MINISTRY OF EDUCATION AND TRAINING

FPT UNIVERSITY

DBS401 COURSE PROJECT REPORT

Developing a PostgreSQL Database Honeypot with AI-Based Attacker Behavior Analysis

Group 3

Phan Thanh Thảo	SE193364	Lecturer:	Hồ Hải
Trần Thị Minh Anh	SE190902	Course:	DBS401
Vũ Trung Hiếu	SE192673		

Ho Chi Minh City

October 2026

ABSTRACT

Database honeypots can show how attackers use a database, but their value depends on what the logs record and on how reliably later analysis labels the recorded queries. This course project studied a locally run PostgreSQL honeypot holding fabricated data. It asked three questions: how completely three logging configurations record simulated attack actions and allow sessions to be rebuilt; whether a rule-based classifier or a locally run language model given a few examples assigns attack stages to individual queries more accurately and points to the supporting queries; and how much both methods lose on attack-technique families withheld from development. All data came from a simulator with known labels. Evaluation used a development set, an independent holdout set of 22 scenarios, and two sets of unseen technique families, with bootstrap intervals computed over scenarios. The most detailed logging configuration recorded all 136 simulated actions, whereas the other two recorded 116 and 121, yet session reconstruction was weak (pairwise F1 score about 0.11) because the simulator used a single source address. On familiar techniques the rule-based method had higher point estimates; at the main setting with two preceding queries, the gap of about 0.05 in macro-averaged F1 score could not be distinguished from zero. Every method lost between about 0.15 and 0.45 macro-averaged F1 score on unseen techniques. The language model lost less than the rule-based method on the first unseen set and in one setting of the second, but there was no evidence that it beat the rule-based method on genuinely new techniques. A hybrid that defers to the language model when the rules recognise nothing helped slightly, at the cost of more false alarms. The data are small and simulated, so the results show trends and not general superiority.

Keywords— PostgreSQL honeypot, logging configuration, session reconstruction, attack-stage classification, few-shot language model, evidence selection, out-of-distribution generalization, bootstrap confidence interval.

ACKNOWLEDGEMENT

The group thanks Mr. Hồ Hải for guiding the DBS401 course project and for feedback on the project proposal.

CONTENTS

Abstract i

Acknowledgement ii

List of Figures vi

List of Tables ix

Abbreviations x

1 Introduction 1

1.1 Background 1

1.2 Problem Statement 2

1.3 Research Objectives 3

1.3.1 RQ1: Logging and Session Reconstruction 3

1.3.2 RQ2: Classification and Evidence 4

1.3.3 RQ3: Generalization to Unseen Technique Families 4

1.4 Significance of the Study 5

1.5 Scope and Limitations 5

1.6 Report Structure 7

2 Literature Review 8

2.1 Review of Previous Studies 8

2.1.1 Scope of the Review 8

2.1.2 Database Honeypots and Attacker Behavior 8

2.1.3 Audit Logging, Session Analysis, and Attack Reconstruction 9

2.1.4 LLM-Based Interpretation and Technique Mapping 11

2.1.5 Classical Machine Learning and LLMs on Structured Security Data . . 12

2.1.6 Generalization to Unseen Attack Behavior . . . 13
2.1.7 Honeypot Realism, Synthetic Decoy Data, and Evaluation . . . 13
2.2 Summary of the Literature Review . . . 14
2.3 Contribution of Research . . . 15

3 Methodology **16**
3.1 Research Design . . . 16
3.2 Data Collection Methods . . . 17
3.2.1 Honeypot environment and logging configurations . . . 17
3.2.2 Behavior labels . . . 18
3.2.3 Scenario generator, ground truth, and the `aid` tag . . . 19
3.2.4 Data sets, authorship, and exposure . . . 20
3.2.5 Preprocessing and classification windows . . . 21
3.2.6 Methods under test . . . 21
3.3 Sampling and Data Analysis Techniques . . . 23
3.3.1 RQ1: coverage and session reconstruction . . . 23
3.3.2 RQ2: classification, evidence, and context . . . 24
3.3.3 RQ3: performance drop on unseen families . . . 25
3.3.4 Uncertainty and multiple comparisons . . . 26
3.4 Limitations of the Methodology . . . 27

4 Experimental and Results **29**
4.1 Introduction . . . 29
4.2 Presentation of Data . . . 29
4.2.1 RQ1: logging coverage and session reconstruction . . . 30
4.2.2 RQ2: classification and evidence on known families . . . 32
4.2.3 RQ3: performance on unseen families . . . 34
4.3 Analysis of Results . . . 39
4.4 Interpretation of Results . . . 40
4.5 Comparison with Literature . . . 40
4.6 Implications of the Results . . . 42

5 Discussion **44**
5.1 Restate the Research Problem or Objectives . . . 44

5.2 Summarize Key Findings 44

6 Conclusion and Future Work **46**

6.1 Conclusion 46

6.2 Future Work 47

References **53**

A Supplementary Result Tables **54**

B Development-Set Results **57**

C Confusion Matrices at $k = 2$ **58**

D Additional Claim Checks **61**

E Original Outputs of the Statistical Scripts **62**

F Reproduction Commands and Seeds **63**

G Prompt Structure **64**

H Audit Findings **67**

LIST OF FIGURES

3.1 Evaluation pipeline shared by the three experiments. 17

LIST OF TABLES

3.1 Logging configurations compared in RQ1 and the gaps hypothesized before the runs. 18

3.2 Behavior labels and representative statements. 18

3.3 Data sets, authorship, and how far they were exposed. 20

3.4 Rubric for characterizing the novelty of a family. Dependency length counts the `depends_on` chain and is unrelated to k. 25

4.1 RQ1: action-level recall by configuration and stage (Dev, 136 actions, 17 scenarios; no interval because the actions are not independent). Number of log records written under each configuration in the last row. 30

4.2 RQ1: recall by gap tag (Dev; an action can carry several tags; counts are actions). 31

4.3 RQ1: share of matched log records that contain each session attribute (Dev). . . 31

4.4 RQ1: pairwise session reconstruction (Dev). Values are identical at thresholds of 1, 5, 15, and 30 minutes (single simulated IP). 31

4.5 RQ2: macro-F1 on the Holdout (22 scenarios, 109 queries, 6 classes; nominal 95% CI; difference paired). 32

4.6 Evidence agreement: mean Jaccard index with ground-truth evidence derived from `depends_on` (evidence only for $k \geq 2$; 95% CI over scenarios, nominal; Holdout 22, OOD-1 9, OOD-2 18 scenarios). 33

4.7 RQ2: paired differences in macro-F1 between values of k on the Holdout (22 scenarios; nominal 95% CI). 34

4.8 H1 (accept when Rule and Qwen agree): coverage, accuracy on accepted windows, Rule accuracy on all windows, and each method's accuracy on the windows H1 sends to review. Accuracy is not macro-F1. Intervals for each value are in the project's results table; the interval is shown for the difference only. . . 35

4.9 Macro-F1 by data set with a fixed class set (95% CI over scenarios, nominal). H4† was computed afterwards from stored labels. OOD-1 has no Noise class, so the Holdout is also scored on the five shared classes. 35
4.10 Drop $\Delta F_1 = F_1(\text{Holdout}) - F_1(\text{unseen})$ on the shared classes; positive means a drop. Fixed-class estimator (main), 1000 resamples, nominal 95% CI. The default-estimator version is in Appendix A. 36
4.11 $D = \Delta F_1(\text{Rule}) - \Delta F_1(\text{Qwen})$; $D > 0$ means Qwen dropped less. Nominal 95% CI and simultaneous max-$|t|$ 95% CI for the family of six comparisons (10 000 resamples, seed 42, $c = 2.592$); p values unadjusted / adjusted. Fixed-class estimator. Comparisons with H4 are exploratory and are in Appendix A. . 36
4.12 OOD-2 split by technique type: absolute macro-F1 difference from the Rule (fixed class set; nominal 95% CI). Post hoc, exploratory subgroups with few scenarios. 37
4.13 Windows labeled Noise by each method, with the number that were truly Noise in parentheses (counts of windows). 38
4.14 Novelty scores (SQL structure, PostgreSQL-specific functions, dependency chain; 1 to 3) and accuracy at $k = 2$ by family. ‡ estimated from the family description, not scored by the builder. n is the number of windows; one rater per family. 38
4.15 Main claims, the evidence for them, and the wording the evidence permits. The complete checklist is in Appendix D. 41

A.1 Sensitivity: ΔF_1 with the default estimator (macro-F1 averaged over the classes present in each resample). Point estimates equal those of Table 4.10. 54
A.2 Differences of drops, all pairs, fixed-class estimator (nominal CI). Rule − Qwen is the registered comparison; those with H4† are exploratory. 55
A.3 Sensitivity: differences of drops with the default estimator. 55
A.4 Absolute macro-F1 differences from the Rule, fixed-class estimator (nominal CI; scenarios in parentheses). H4† was computed after the fact. 56

B.1 Prompt ablation for Qwen3 8B on Dev (macro-F1). A: same examples at every k; B: balanced examples added; C: CONTEXT/TARGET markers and target-only instruction; D: reason before label; E: stricter label definitions; F: extra Collection examples. The first prompt is the starting point; the rule-based method is the reference. 57
B.2 Sensitivity of Qwen to corrupted context on Dev (macro-F1 and difference from ideal context; 95% CI). The "sessionizer" condition has only two mixed sessions. 57

C.1 Confusion matrix, Holdout, Rule, $k = 2$. 58
C.2 Confusion matrix, Holdout, Qwen, $k = 2$. 58
C.3 Confusion matrix, Holdout, H4†, $k = 2$. 58
C.4 Confusion matrix, OOD-1, Rule, $k = 2$. 59
C.5 Confusion matrix, OOD-1, Qwen, $k = 2$. 59
C.6 Confusion matrix, OOD-1, H4†, $k = 2$. 59
C.7 Confusion matrix, OOD-2, Rule, $k = 2$. 59
C.8 Confusion matrix, OOD-2, Qwen, $k = 2$. 59
C.9 Confusion matrix, OOD-2, H4†, $k = 2$. 60

D.1 Additional claims and permitted readings. 61

ABBREVIATIONS

Acronym	Meaning
`aid`	Opaque action identifier written as an SQL comment `/*aid=<id>*/` so that a log record can be matched to the scenario step that produced it
C0, C1, C2	The three logging configurations compared in RQ1 (Section 3.2.1)
CI	Bootstrap 95% confidence interval, resampled over scenarios
D	Difference between two performance drops, $D = \Delta F_1(\text{Rule}) - \Delta F_1(\text{Qwen})$ (Section 3.3)
DBMS	Database management system
ΔF_1	Drop in macro-averaged F1 from the Holdout set to an unseen-family set; positive means the score fell
H1, H3, H4	Strategies that combine the rule-based and language-model outputs (Section 3.2.6)
Holdout	Independent test set of known-family scenarios, scored once
IP	Internet Protocol (source address)
JSON	JavaScript Object Notation
k	Number of preceding queries in a classification window
LLM	Large language model
macro-F1	Unweighted mean of per-class F1 scores
MITRE ATT&CK	Knowledge base of adversary tactics and techniques; the project labels are aligned with its concepts
OOD-1, OOD-2	First and second sets of attack-technique families withheld from method development
pgAudit	PostgreSQL audit logging extension
Qwen	Qwen3 8B model with the frozen prompt B+C+E, served locally through Ollama
RQ, RO	Research question, research objective
Rule	The rule-based classifier
SQL	Structured Query Language

CHAPTER 1

INTRODUCTION

1.1 Background

Database management systems are part of the critical data layer of modern applications. When a database service is exposed through a public network, an attacker may probe its identity and configuration, attempt to authenticate, issue queries to discover or collect data, and try to alter database state or use database capabilities to reach other resources. Unlike a single isolated query, these actions can form a sequence in which later steps depend on information or access obtained earlier. Understanding such behavior therefore requires both reliable event capture and analysis that preserves the relationships among events.

A database honeypot provides a controlled way to observe activity directed at a decoy service. Song, Smaragdakis, and Griffioen deployed 278 honeypots over 20 days, including low-interaction emulations of PostgreSQL and other database systems, and reported persistent brute-force attempts together with medium- and high-interaction exploitation that manipulated data, copied and deleted it, or used the DBMS to reach the underlying system [1]. Earlier work demonstrated how a database honeypot can be built on open-source MySQL and used together with honeytokens for deception [2]. These studies establish the value of database-focused deception, while also motivating careful measurement of what a particular database deployment can observe.

The raw trace is the foundation for downstream analysis. A logging configuration that captures statements but omits relevant connection, error, or parameter information may leave gaps in the account of an interaction. Conversely, highly detailed logging can increase storage use and query-processing overhead, and may expose sensitive values if real data is used. Even when individual events are recorded, analysts still need to determine which events belong to the same logical session and which queries provide evidence for a particular attack stage.

Recent work has also explored large language models for interpreting security alerts and mapping low-level observations to higher-level attack behavior. Schärmer et al. evaluate ChatGPT and Gemini for classifying IDS alerts and report that carefully chosen few-shot examples and system context support more effective classification, while noting limitations related to cost, reliability, and sensitive-data exposure [3]. Meng et al. present RHINO, which maps network logs to adversarial tactics and techniques; they argue that single-step LLM classification often hallucinates technique names and produces decontextualized mappings, and they propose guided reasoning as an alternative [4]. These findings motivate testing contextual classification and evidence selection on database query traces, rather than assuming that a model label alone is a reliable explanation.

1.2 Problem Statement

Despite the relevance of honeypots and language-model-assisted security analysis, three practical questions remained for a PostgreSQL honeypot. First, logging configurations may differ in their ability to capture the actions needed to identify seven project-defined behavior classes: Discovery, Credential Access, Privilege Escalation, Collection, Exfiltration, Impact, and Noise. Six of these classes describe attack stages and one describes benign activity. The project had to quantify these differences and determine whether the resulting records contain the fields required for later analysis. Second, captured log lines may not directly identify a complete attack session. Sessionization based on an inactivity threshold can merge unrelated activity or split a real multi-step sequence, and its accuracy must be measured against known scenario ground truth. Third, query-level classification is difficult when the meaning of a query depends on earlier events. Rule-based methods may be transparent but brittle to variations; a few-shot language model may use context more flexibly but can produce unsupported labels or evidence identifiers. Neither approach should be considered reliable without measuring both classification and evidence quality.

A further challenge is generalization. A method can perform well on variations that resemble its development scenarios while failing on a technique family that was not used to write its rules or examples. To assess this distinction, the evaluation needed a controlled split by scenario family, with held-out families kept isolated from method development, and a comparison of the query evidence used to justify labels as well as the labels themselves. Without this separation, apparent performance may reflect leakage or familiar patterns rather than the ability to analyze

unseen behavior.

Accordingly, this project built a reproducible, local evaluation pipeline that connects PostgreSQL logging, session reconstruction, query classification, and evidence assessment, and used it to answer the research questions below. The project measures the suitability of captured interaction traces for behavior analysis; it does not claim that synthetic scenarios represent the full diversity of real-world attackers.

1.3 Research Objectives

The project was guided by the following overarching question: *To what extent can a high-interaction PostgreSQL honeypot combined with behavior analysis record, identify, and explain attack stages from interaction logs, including when attackers use previously unseen techniques?* The three research questions form one chain: RQ1 selects the logging configuration whose logs feed RQ2, and the methods frozen in RQ2 are the ones tested on unseen techniques in RQ3.

1.3.1 RQ1: Logging and Session Reconstruction

RQ1. Which PostgreSQL honeypot logging configurations miss or mis-reconstruct which types of attack actions and sessions, to what extent, and which configuration is sufficient for subsequent behavior analysis?

RO1.1: Measure action-level recall for each attack and benign behavior class across three logging configurations (C0, C1, C2), and break it down by stage and by the log-visibility tags ("gap tags") attached to each action.

RO1.2: Assess which of the six session attributes—user, source IP address, database, query, result, and parameter—are present in parsed log records, reported field by field, and evaluate session reconstruction using pairwise precision, recall, and F1 over inactivity thresholds of 1, 5, 15, and 30 minutes.

RO1.3: Select the minimum configuration that satisfies the criterion applied in this project (recall of at least 95% overall and for every stage, a threshold chosen by the group and not a standard), considering capture coverage, field completeness, and log volume measured as the number of log records.

1.3.2 RQ2: Classification and Evidence

RQ2. Between a rule-based method and a locally run few-shot large language model, which more accurately assigns an attack stage to each query when preceding query context is available, and how closely does each method's selected evidence match the known evidence?

RO2.1: Compare classification performance and technical output reliability using macro-averaged F1, per-class F1, a confusion matrix, the invalid-output rate (outputs that cannot be parsed or violate the required JSON schema, counted separately), and, for the language model, the rate of fabricated evidence identifiers, meaning identifiers that do not correspond to any query in the window.

RO2.2: Evaluate predicted query evidence against dependency-derived ground truth. The Jaccard index is the primary measure; precision, recall, and F1 of the evidence sets are reported in addition. Manually review a 20% sample to assess ground-truth quality.

RO2.3: Compare context windows containing the current query and 0, 2, or 4 preceding queries, measuring classification, evidence quality where context is available, and inference time.

1.3.3 RQ3: Generalization to Unseen Technique Families

RQ3. How much do the classification and evidence-matching performance of the two methods decrease on attack-technique families excluded from method development, compared with variations of known families?

RO3.1: Measure the change in macro-averaged F1 between the known-family evaluation set and each isolated unseen-family set, and describe the corresponding change in evidence agreement.

RO3.2: Analyze per-class confusions, the number of windows labeled Noise, and fabricated evidence identifiers on unseen families.

RO3.3: Examine how SQL structural novelty, PostgreSQL-specific functions or extensions, and the complexity of inter-query dependencies relate to performance decline, as a qualitative description.

The attack labels are a project-defined taxonomy aligned with relevant MITRE ATT&CK concepts; they are not presented as a verbatim set of official MITRE ATT&CK tactics. Each

scenario step carried an explicit stage label and dependency links so that action, session, and evidence ground truth could be generated consistently. Only RQ1 uses all seven labels; RQ2 and RQ3 classify six (Section 3.2.2).

1.4 Significance of the Study

The project has methodological and practical significance. Methodologically, it evaluates the entire path from database interaction to an interpretable query-level result. It separates logging coverage from session reconstruction, and separates label accuracy from evidence quality. This makes it possible to identify whether an analysis error originates in missing telemetry, incorrect session grouping, stage classification, or evidence selection. The use of scenario-family grouping and isolated held-out families, together with intervals computed over scenarios, was intended to limit query-level leakage and to make the uncertainty of small samples visible.

Practically, the study shows limits that matter to anyone building a similar system: complete action-level logs were not enough to reconstruct sessions in this setting; a rule-based method and a locally run model could not be separated on familiar techniques at the main setting; and both lost accuracy on unfamiliar techniques. The local deployment and fabricated data supported repeatable experiments without paid inference services or exposure of real organizational data. The results are evidence about the controlled test environment and not an estimate of detection rates in production networks.

1.5 Scope and Limitations

The project was limited to a locally run PostgreSQL honeypot in Docker, populated with fabricated data and sensitive-looking decoy tables. The environment was not exposed to the public Internet. A parameterized Python scenario generator varied query structure, object names, order, client or tool characteristics, and timing jitter. Scenarios were informed by attack behaviors reported in the reviewed literature, including credential guessing, post-authentication data access, and program-execution features reported by Song et al. [1] and post-exploitation credential discovery described by Gómez-López et al. [5]. Their exact query sequences and dependency links are synthetic ground truth and not replicas of observed campaigns.

Logging configurations

RQ1 compared three logging configurations. C0 sets `log_statement = all`. C1 enables pgAudit session logging. C2 extends C1 with `log_connections`, `log_disconnections`, `log_min_error_statement`, and `pgaudit.log_parameter`. A fourth configuration based on a proxy or packet capture was considered at the outset and is out of scope. Parsed records were normalized to a shared schema and sessions were grouped by source IP, user, and an inactivity threshold.

Classification and evidence

RQ2 compared a rule-based classifier with a locally hosted language model of about eight billion parameters (Qwen3 8B), served through Ollama. Both methods received consistently normalized query windows. The model returned JSON containing a label and a list of evidence identifiers; the schema was requested in the prompt and checked afterwards and was not enforced during decoding, so that fabricated identifiers could be measured instead of being ruled out by construction. Evidence ground truth consisted of the target query and its declared dependencies that remained within the window.

Limitations that apply to every result

- All data are simulated, including the benign Noise traffic. The scenarios are based on Song et al. [1] and on a vendor report; they are not recordings of real attackers.
- The samples are small: 102 development queries, 109 Holdout queries, and two sets of unseen families with 57 and 145 queries from only three and six families. Confidence intervals are wide, and a difference smaller than its interval supports no conclusion.
- The simulator ran from a single source address, so session grouping could not be tested realistically. In the development and Holdout sets the context of each window was taken from the true scenario identifier (ideal context).
- Isolation of the test data was not complete: Claude read part of the Holdout content while checking for errors, and the SQL of the first unseen set (OOD-1), which Claude had written before the methods were frozen, appeared in the working conversation after the freeze (it was not used to adjust anything). The effect of these two events was not measured; the

group accepted the risk, which is a decision and not a measurement.

- The language-model prompt was tuned on the development set, so development scores are optimistic.
- Some scenario steps failed for environmental reasons (for example `COPY ... PROGRAM` commands that reach outside the container, an extension that was not loaded, or a `LOCK` outside a transaction); the statements were still logged and matched.
- The manual evidence review used one reviewer, who was an AI (Claude) and not a member of the group.
- Not done: CPU and memory measurement during inference, overhead measurement with pgbench, and a "log evasion" scenario group. Out of scope: the proxy or packet-capture configuration and experiments with several source addresses.
- Some thresholds are the group's own choices and were not confirmed as standards: the definition of a "light" configuration, the 95% recall criterion and whether it applies to the total or to each stage, and the meaning of "sufficient attributes".

1.6 Report Structure

This course project report follows the six-part structure of the course guide. Chapter 1 introduces the problem, objectives, significance, and scope. Chapter 2 reviews related work. Chapter 3 describes the methodology: design, data, analysis techniques, and limitations. Chapter 4 presents and interprets the experimental results for RQ1 to RQ3. Chapter 5 discusses the findings and their limits, and Chapter 6 concludes and proposes future work. The appendices contain the supplementary tables, the audit findings, the claim checklist, the original outputs of the statistical scripts, and the reproduction commands.

CHAPTER 2

LITERATURE REVIEW

2.1 Review of Previous Studies

2.1.1 Scope of the Review

This review examines research that supports the main stages of this project: database honeypot deployment and attacker behavior, audit logging and attack reconstruction, learning-based interpretation of security data, generalization to unseen attack behavior, and the evaluation of honeypot realism. The reviewed studies differ in platform, interaction level, data type, and evaluation setting. Their results therefore inform the design of the project, but they do not predict performance on its controlled PostgreSQL query traces. Where a source was available only as a title or a short abstract, the description below is limited to what that source states.

2.1.2 Database Honeypots and Attacker Behavior

Song, Smaragdakis, and Griffioen deployed 278 honeypots over 20 days in March–April 2024 [1]. Their 220 low-interaction honeypots emulated MySQL, MSSQL, PostgreSQL, and Redis and revealed relatively low scanning activity but persistent brute-force attempts. Their 58 medium- and high-interaction honeypots revealed three types of exploitation: direct manipulation of the database, ransom-driven copying and deletion of data, and use of the DBMS as a route to take over the underlying system. This is the most directly relevant empirical study for the project, because it covers PostgreSQL and multi-step behavior. Its deployment was Internet-facing and differs from the local controlled experiment conducted here, so its observed volumes and frequencies are not treated as expected project results.

Rocha and Pereira used honeypot tools to characterize attacks on relational DBMSs and reported that the MySQL DBMS was the most targeted system among those analyzed [6]. Their study examines MySQL, PostgreSQL, and Oracle decoys, but its target prevalence results are

specific to their deployment and do not transfer to PostgreSQL in the present setting. Its value for this project is as motivation for DBMS-specific measurement.

Wegerer and Tjoa survey database honeypots and honeytokens and describe a MySQL honeypot built with open-source components, including an audit plugin that records queries and login attempts [2]. The work shows that database audit mechanisms are a central design concern for a database honeypot, although its implementation is MySQL-specific and its audit plugin cannot be assumed to apply unchanged to PostgreSQL.

Work focused on PostgreSQL and MS SQL honeypots treats post-authentication interaction as the object of capture. Al Hafidz et al. study enhancing detection in a PostgreSQL shell honeypot through interactive query responses [7]. Karo Karo et al. study capturing post-login behavior in MS SQL honeypots through a protocol-aware TDS relay [8]. Both titles indicate that what happens after authentication is the central capture problem, which is the concern addressed by RQ1. Their full evaluation results were not available for this review, so no quantitative comparison is drawn from them.

Gómez-López et al. present SQLiAgent, a multi-agent framework for SQL injection auditing whose LLM-assisted post-exploitation mode interprets database schemas, identifies credential tables, and generates privilege-escalation SQL statements [5]. The work is offensive and is validated on the OWASP Broken Web Applications testbed, but it documents the behaviors that the project's Credential Access and Privilege Escalation scenarios must represent. It also shows that LLM-assisted tools can produce such statements from inferred schema structure, which is relevant to the threat model.

2.1.3 Audit Logging, Session Analysis, and Attack Reconstruction

Database audit trails provide structured records that can support analysis beyond individual query inspection. Zimniak, Getta, and Benn transform database audit-trail information to discover periodic patterns in workload activity [9]. Their work concerns workload prediction rather than security detection, but it demonstrates the value of preserving ordering and timing when analyzing sequences of database operations.

Biswas and Niemi examine how auditing requirements determine which database logs must be collected [10]. They identify a practical trade-off: insufficient logging prevents reliable auditing, while redundant logging adds cost and overhead on database performance. This motivates the project's comparison of logging profiles using action capture, required-field complete-

ness, and log volume. Zeng, Xiao, and Chen similarly study auditing overhead and adaptation in the Linux audit framework and propose adaptive logging that changes the event types and frequencies collected [11]. Although their work is not specific to PostgreSQL, it indicates that instrumentation overhead is a relevant cost of detailed logging; this project measured log volume only and lists overhead measurement as an open limitation.

Choi and Lee analyze the internal structure of the Microsoft SQL Server transaction log in unallocated disk space [12]. Their analysis indicates that INSERT, UPDATE, and DELETE queries correspond one-to-one with log records, whereas CREATE and DROP TABLE are represented by combinations of several records within a single transaction. For this project the finding is a warning for parser design: a single query may produce several log records, and some statements may produce none at all, so the parser must not assume a fixed correspondence between queries and records.

Goseva-Popstojanova and colleagues characterize malicious web traffic captured by high-interaction honeypots that ran fully functional three-tier web systems [13]. They describe malicious sessions with 43 session attributes, including duration, number of requests, and bytes transferred, and show that supervised learners separated attack sessions from vulnerability-scan sessions with high detection and low false-alarm rates. Their use of session-level attributes motivates the session features and session-level evaluation in RQ1, although their domain is HTTP rather than database queries.

Landauer et al. introduce CAM-LDS, a labeled log dataset of seven attack scenarios covering 81 techniques across 13 tactics, collected in a fully open-source and reproducible environment [14]. Their illustrative study uses an LLM to interpret the logs, and reports that correct attack techniques were predicted perfectly for about one third of attack steps and adequately for another third. The result suggests that LLM-based log interpretation is partially successful and that a substantial share of steps remains difficult, which supports reporting per-class results and not only aggregate scores in this project.

SLEUTH reconstructs attack scenarios in real time from commercial off-the-shelf audit data using a main-memory dependency graph and tag-based techniques for source identification and impact analysis [15]. TraceBack likewise reconstructs attack scenarios from provenance graphs, using causal analysis and sequence modeling to reduce dependency explosion and false positives [16]. These systems operate on host and provenance data rather than database query traces, but they support modeling events as dependent steps. Zambianco et al. further show that using

execution preconditions and effects of ATT&CK techniques to model attack paths allows decoy selection to account for the logical dependencies between attack steps [17]. This provides the conceptual basis for the depends_on annotations used to generate evidence ground truth.

2.1.4 LLM-Based Interpretation and Technique Mapping

Schärmer et al. evaluate LLMs for interpreting IDS alerts, including distinguishing attack-related alerts from false positives and associating alerts with attack techniques [3]. They examine how additional logs, configuration data, and few-shot examples affect classification performance and consistency, and conclude that carefully chosen examples and system context support effective classification. Their inputs are network and host alerts rather than SQL statements, so the findings must be validated on query data before they are assumed to apply here.

Meng et al. present RHINO, which maps network logs to adversarial tactics and techniques through guided reasoning [4]. Their abstract reports that existing LLM-based methods frequently hallucinate technique names or produce decontextualized mappings because they classify in a single step, and that these failures motivate a reasoning-based approach. Meng et al. also present CYBERTEAM, a benchmark that organizes threat hunting into a standardized workflow of 30 tasks and 9 operational modules, and report that this structure improves results over open-ended reasoning strategies such as in-context learning, chain-of-thought, and tree-of-thought prompting [18]. Both works motivate testing whether structured context improves query-level classification and whether the selected evidence is actually supported by logged events.

Yang et al. propose APTREC, which uses an LLM to recognize APT tactics and techniques from attack subgraphs through a two-stage architecture: a technique recognition module and a tactic reasoning module [19]. They report a 13% F1-score improvement and a 9% recall increase over representative existing methods, together with explainability. Its two-stage design is a useful contrast to the single-step prompt used in RQ2, although its subgraph input differs from the query windows used here.

Wang et al. study zero-shot multi-label identification of attack techniques in which LLM reasoning is augmented with semantic evidence discovery [20]. Its relevance to RQ2 lies in the explicit linkage between predicted labels and supporting evidence. The abstract was not available for this review, so the description is limited to the title.

Neves et al. compare zero-shot LLMs with rule-based methods for clinical phenotyping

from synthetic patient summaries [21]. GPT-4o achieved a recall of 0.97 and a macro-F1 of 0.92, whereas the rule-based method was highly precise (0.92) but had a recall of only 0.36. The authors also show that combining rule-based and LLM methods can direct manual review to discordant cases. This is the closest published analogue to the RQ2 design: it compares the two approaches on a structured classification task, and its result that rules and LLMs fail in complementary ways suggests that the rule-based and LLM outputs should be analyzed jointly rather than only ranked.

Duan et al. survey few-shot learning for intrusion detection and discuss its potential where labeled examples are scarce, together with open challenges in applying it reliably [22]. The proposed project uses few-shot prompting of a fixed local model rather than training a new few-shot learning method, so the survey serves as background on low-example security classification rather than as a method to replicate.

2.1.5 Classical Machine Learning and LLMs on Structured Security Data

Liu and Lang survey machine-learning and deep-learning methods for intrusion detection, covering taxonomy, algorithms, datasets, and the challenges of detection accuracy, false-alarm rates, and unknown attacks [23]. Alharthi et al. compare classical machine-learning and deep-learning models on binary and multiclass intrusion datasets and find that the strongest model depends on both the dataset and the classification task [24]. These studies support reporting macro-F1 and per-class results and including transparent baselines. Their feature spaces differ from SQL query logs, so their numerical results cannot be transferred to the proposed task.

Rehman et al. serialize network flows into compact text and study zero-shot, instruction-guided, and few-shot prompting with grammar-constrained outputs and a calibrated decision threshold [25]. They find that unguided prompting is unreliable and that instructions plus boolean flags substantially improve detection, while decision quality decreases as the evaluation set grows from 200 to 2000 flows. Tabular baselines remain more stable and faster. Mehavilla et al. benchmark LLMs against machine-learning and deep-learning models on Zeek logs and report that LLMs achieve F1-scores above 95% but do not outperform the best classical model, XGBoost, which reaches 96.96% while using only 4% of available CPU [26]. Together these studies caution against assuming that an LLM is preferable for structured classification, and they motivate a direct comparison under identical inputs and splits.

2.1.6 Generalization to Unseen Attack Behavior

Generalization is the central concern of RQ3. Lust and Condurache survey methods for assessing whether a deep neural network's output is reliable for a given input, grouping predictive uncertainty, out-of-distribution detection, and adversarial example detection within the broader question of generalization performance [27]. Their framing supports the project's distinction between variations of known technique families and unseen families, and suggests that out-of-distribution behavior should be measured rather than assumed. Zhou and Wang report that transformer networks face difficulty generalizing to out-of-distribution data and propose GROD, which uses synthetic outliers and penalizes out-of-distribution misclassification during training [28]. The method is designed for neural training rather than for prompted LLMs, so it does not directly apply here, but it confirms that transformer generalization to unseen distributions is a recognized problem. Guo et al. address out-of-variable generalization for discriminative models [29]; only the title of this work was available for this review, so no finding is drawn from it.

The scarcity of labeled multi-stage data is a related obstacle. Das and Nair publish a dataset of normal smart-home traffic and seven multi-stage attack scenarios, motivated by the observation that multi-stage attacks are more difficult to detect [30]. Their domain is network flows, but the paper illustrates why the project must generate its own labeled, dependency-annotated query scenarios and document the generation process in detail.

2.1.7 Honeypot Realism, Synthetic Decoy Data, and Evaluation

A honeypot's value depends on the depth and quality of the interactions it sustains. Nunez Migliorisi proposes telemetry-based measures of realism and deception effectiveness, including attacker dwell time, interaction depth, behavioral diversity, and malware family richness, and argues that raw connection counts are insufficient [31]. Ferguson-Walter et al. analyze a controlled experiment with more than 130 professional red teamers and compare measures of forward progress across four conditions defined by the presence of decoys and whether attackers were informed about deception. They report that the combination of decoys and information about deception had the greatest impact on attacker behavior relative to a control condition [32]. These works support using multiple indicators of engagement, although the current project primarily evaluates observability and analysis quality in controlled scenarios.

Synthetic decoy values also need validation. Weinberg proposes PHANTOM, which gen-

erates honeytokens conditioned on organization-specific naming and value distributions and evaluates them with a believability score combining syntactic validity, semantic coherence, statistical plausibility, and human acceptance [33]. The same decomposition can guide a plausibility check on the synthetic sensitive tables in this project, although the project's records are not honeytokens and the score was not reproduced in full. Msaad et al. show that an advanced attacker can fingerprint most common open-source honeytokens without triggering an alert [34], which is a limitation the project must acknowledge.

Adaptive and LLM-based deception show the direction of recent work. Kouremetis et al. present Mirage, an emulation and simulation system for testing deceptions against adversaries that use AI search and planning [35]. Wushishi et al. propose D3O-IIoT, a reinforcement-learning approach that coordinates honeypot deployment, moving-target defense, and fake telemetry in industrial IoT networks [36]. Safargalieva et al. present OHRA, an LLM-driven honeypot that supports multiple protocols, with SSH, Telnet, and HTTP fully interactive and FTP, SMTP, IPP, and SNMP partially implemented [37]. Bridges et al. systematize research on honeypots and LLMs, providing a taxonomy of detection vectors, a canonical architecture for LLM-powered honeypots, an evaluation framework, and a discussion of log analysis as a route to automated intelligence [38]. Their review highlights the shortage of realistic labeled data and the risk that intelligent adversaries detect LLM-driven behavior. The present project uses an LLM to analyze recorded query traces rather than to generate responses, so response fidelity is not an outcome measure, but the review supports explicit evaluation criteria and cautions against treating LLM output as self-validating evidence.

2.2 Summary of the Literature Review

The reviewed database honeypot studies show that DBMS-specific decoys can expose persistent authentication attempts and deeper multi-step behavior, and that the interaction level and the capture mechanism determine what can be observed [1], [7], [8]. Audit-trail and reconstruction studies show why event order, timing, dependencies, and field coverage matter, while also identifying logging overhead as a relevant cost [10], [11], [12]. Research on LLMs for security data provides useful baselines, contextual prompting strategies, evidence-oriented designs, and evaluation practices, but its results on network flows, alerts, and clinical text do not establish performance on SQL query traces [3], [4], [21], [25]. Generalization studies motivate isolating unseen families rather than relying on random splits [27], and honeypot evaluation studies show

that engagement and realism require more than counting connections [31], [32].

A gap remains at the intersection of these areas. The reviewed sources do not establish which PostgreSQL logging profile is sufficient to reconstruct labeled query sessions in a controlled honeypot, nor whether a local few-shot LLM improves query-stage classification and evidence selection over a rule-based method when evaluated on technique families withheld from development. This project addresses that gap by evaluating capture coverage, session reconstruction, classification, evidence quality, and generalization in a single reproducible pipeline. The conclusions of this report apply to the tested configuration and scenarios, not directly to all production databases or real-world attackers.

2.3 Contribution of Research

The project delivers an integrated and reproducible evaluation of a local PostgreSQL honeypot and its behavior-analysis pipeline. Specifically, it:

C1: compared three logging configurations using action-level capture recall, field-by-field completeness of the required attributes, and log volume in records;

C2: measured query-session reconstruction across inactivity thresholds against labeled scenarios with explicit dependencies between attack steps, and showed that single-address simulation makes this measurement uninformative;

C3: compared a rule-based classifier with a locally hosted few-shot language model on the same query windows, reporting macro-F1, per-class performance, invalid outputs, fabricated evidence identifiers, and evidence agreement, with uncertainty over scenarios;

C4: tested both methods, unchanged, on two sets of technique families excluded from development, with a difference-of-drops measure, an adjustment for multiple comparisons, and an explicit account of which parts of the design were chosen after seeing results and how far the test data were isolated.

These are findings about a controlled simulated environment. The study uses them to describe logging and analysis trade-offs, and to provide a basis for the future extensions listed in Chapter 6.

CHAPTER 3

METHODOLOGY

3.1 Research Design

The study is a quantitative, controlled-environment evaluation organized into three experiments, each answering one research question. RQ1 measured how well three PostgreSQL logging configurations capture attack actions and reconstruct sessions. RQ2 compared a rule-based classifier with a locally hosted few-shot large language model on query-level attack-stage labels and evidence selection. RQ3 measured how much the performance of both methods declined on attack-technique families that were excluded from their development. The experiments form a chain: RQ1 selected the configuration (C2) whose logs were the input of RQ2; the methods frozen in RQ2 were then applied without change in RQ3; and the weak session grouping found in RQ1 is the reason why the RQ2 context had to be stress-tested.

All three experiments share one pipeline (Figure 3.1) so that differences between results can be attributed to the variable under study rather than to inconsistent data handling. A parameterized scenario generator produces attack and benign query sequences together with ground-truth labels and dependencies; the sequences are executed against a local PostgreSQL honeypot under a chosen logging configuration; the logs are parsed into a common event schema and each record is matched to its scenario step through the `aid` tag; for RQ1, parsed events are grouped into sessions; for RQ2 and RQ3, the events are arranged into query windows and classified; and the predictions are compared with the ground truth using the metrics defined in Section 3.3.

Four data sets play different roles (Table 3.3). The Dev set was used to choose rules, prompts and combination strategies and could be run freely. The Holdout set tested RQ2 and was scored once. Two sets of unseen technique families (OOD-1 and OOD-2) tested RQ3 and were each scored once. The methods were frozen before any test scoring: the rule-based classifier, and the language model Qwen3 8B with prompt B+C+E, seed 42, and temperature 0. The combination

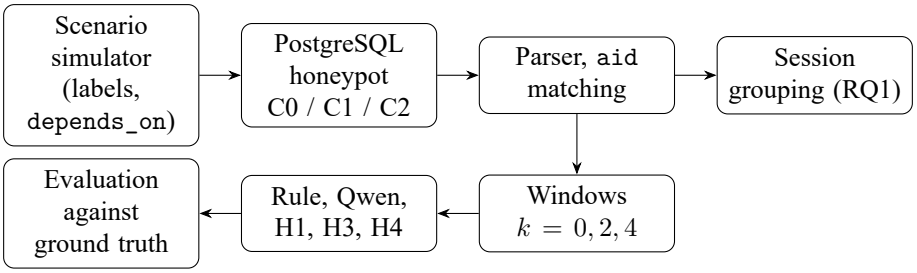


Figure 3.1: Evaluation pipeline shared by the three experiments.

strategies H1 and H3 were specified on Dev before the Holdout was scored. H4 was conceived after OOD-1 had been scored, and its definition was fixed before OOD-2 was scored. The project was therefore not pre-registered as a whole, and only OOD-2 is a clean test of H4.

3.2 Data Collection Methods

3.2.1 Honeypot environment and logging configurations

The honeypot is a PostgreSQL 15 server (official Docker image with the pgAudit package added) running in Docker on a local machine, writing JSON-format logs. It is not exposed to any public network. Its databases contain fabricated data, including decoy tables whose names and contents resemble sensitive records such as user accounts and payment information. No real personal or organizational data is used. Each configuration was run separately from a clean database, using the same workload: the Dev scenario set (136 actions).

RQ1 compares three logging configurations, summarized in Table 3.1. The expected gaps were hypotheses stated before the runs. The runs refuted part of them: all three configurations recorded failed statements and failed logins, because `log_min_error_statement` defaults to `error`. The gaps that were actually found are reported in Section 4.2.1. In all three configurations pgAudit (where loaded) logs every statement class (`pgaudit.log = all`, including catalog queries), C0 and C1 leave connection logging off, and `log_min_error_statement = error`, which is the PostgreSQL default, is set identically in all three (C2 only states it explicitly). Failed logins are written by the server as errors in every configuration.

The six session attributes are user, source IP, database, query, result, and parameters. RQ1 checked, for each configuration and field, how often the attribute is present in the parsed records that match a scenario action. The attribute check is reported field by field; no composite "sufficient attributes" flag was computed.

Table 3.1: Logging configurations compared in RQ1 and the gaps hypothesized before the runs.

Config.	Settings	Hypothesized gaps	Finding (Section 4.2.1)
C0	`log_statement = all`	Statements that fail to parse and failed logins not recorded; prepared-statement parameters not recorded	Failed statements and logins were recorded; prepared statements 54.5% and multi-connection actions 36.8% recorded
C1	pgAudit session logging	Connection and disconnection events and prepared-statement parameters not recorded	Same gaps as C0 for multi-connection actions (36.8%); prepared statements fully recorded
C2	C1 plus `log_connections`, `log_disconnections`, and `pgaudit.log_parameter`	Expected to capture all six session attributes	All 136 actions recorded; parameters present in only 3.7% of records

3.2.2 Behavior labels

Each action is assigned one of seven labels, six describing attack stages and one describing benign activity. The labels are a project taxonomy aligned with the concepts of the MITRE ATT&CK framework; they are not a verbatim copy of its tactics. Table 3.2 defines each label. Persistence actions such as creating a backdoor role are labeled Impact.

Table 3.2: Behavior labels and representative statements.

Label	Definition	Representative statements
Discovery	Probing the structure of the database and server	Queries on `information_schema`, `pg_catalog`, `version()`
Credential Access	Obtaining or guessing account credentials	Reading credential tables; repeated failed logins
Privilege Escalation	Gaining higher database privileges	Appears in the data only as connection events without SQL (RQ1 only)
Collection	Reading sensitive data in bulk	`SELECT` over sensitive tables, large result sets
Exfiltration	Moving data outside the database	`COPY ... TO PROGRAM`, `lo_export`, `dblink` to another server
Impact	Altering or destroying data or structure, or running operating-system commands	`DROP TABLE`, `UPDATE`, `DELETE`, `COPY ... FROM PROGRAM`, creation of backdoor roles
Noise	Ordinary queries that are not part of an attack	Routine application-style reads and writes

Privilege Escalation events in the collected data are connection events and have no SQL statement to classify. RQ1 therefore uses seven labels, whereas RQ2 and RQ3 classify the six labels that have SQL statements; the four Privilege Escalation actions of OOD-1 lie outside the

scope of RQ2 and RQ3. Statements that invoke program-execution features are assigned by a rule fixed before scenario generation: `COPY ... FROM PROGRAM` is Impact because it changes system state by running operating-system commands, a documented PostgreSQL attack pattern [1], and `COPY ... TO PROGRAM` is Exfiltration. Any statement that the labeling rules could not assign unambiguously was recorded in the ground-truth documentation and not resolved by the classifier.

3.2.3 Scenario generator, ground truth, and the `aid` tag

A Python generator produces attack and benign sessions from templates. Each template is a sequence of steps, and every step declares a stage label, a unique step identifier, a `depends_on` list naming the earlier steps whose results it uses, and zero or more gap tags. The seven gap tags describe how an action appears in the log and are independent of the stage: `error_query`, `auth_failure`, `nested_dynamic`, `prepared_statement`, `pg_specific`, `log_evasion`, and `multi_connection`. The stage answers what the attacker wants; the tag answers how the log sees it. Because these fields are declared in the template, the ground truth is generated automatically and not annotated afterwards. The generator varies table and column names, step order where allowed, client tool, literal values, and timing jitter.

To connect log records with ground truth, the simulator attaches an opaque comment `/*aid=<id>*/` to every statement. The tag is a random 12-character hexadecimal string and encodes neither stage nor order. The parser extracts it into a separate column and removes it from the statement before the data reach RQ2. Statements run inside `DO` or `EXECUTE` blocks are logged by pgAudit without the tag of the parent command, so the parser lets them inherit the `aid` of the parent; after parsing, records that carry the same `aid` are collapsed to the first one, so each action yields one record. Events without SQL, such as failed logins, are matched by time, source IP and user. Log lines without an `aid` (health checks, background processes) are marked as unmatched and excluded from RQ2 and are not treated as Noise. Benign Noise sessions are generated from routine query patterns and include statements that resemble Discovery (for example a normal client running `\d`) and harmless errors, so that the classifiers had to separate attacks from ordinary activity.

3.2.4 Data sets, authorship, and exposure

Scenarios are grouped into families, a family being a set of scenarios with the same main technique and target. The Dev set has six families: `recon_sweep`, `cred_probe`, `data_heist`, `exfil_impact`, and `full_chain` (three variants each) and the benign `noise_app` (two variants). Table 3.3 lists the four data sets. Descriptions of the unseen families are limited to their technique, and no SQL from the Holdout or the unseen sets appears in this report, because these sets are sealed.

Table 3.3: Data sets, authorship, and how far they were exposed.

Set	Role	Size	Classes	Author	Exposure
Dev	Choosing rules, prompts, strategies; run freely	17 scenarios; 102 queries (136 actions in RQ1)	6	Group and Claude	Scores are optimistic. Includes the earlier test fold (6 scenarios) that was exposed during language-model debugging
Holdout	RQ2 test, scored once	22 scenarios; 109 queries (138 steps)	6	Separate agent	Not fully blind: Claude read part of the content and test output once printed fragments of SQL. Few-shot examples come from Dev, with a test that blocks identical SQL
OOD-1	RQ3, exploratory	9 scenarios, 3 families; 57 queries (63 actions)	5 (no Noise; one Credential Access query)	Claude, before the freeze	SQL appeared in the working conversation after the freeze. Not an independent test
OOD-2	RQ3, later test	18 scenarios, 6 families; 145 queries (159 actions)	6 (Discovery 21, Collection 27, Exfiltration 12, Impact 52, Credential Access 6, Noise 27)	Separate agent, after OOD-1 results	Partly exposed: error messages of the run checker printed the program strings of `COPY ... PROGRAM` steps

OOD-1 contains three families: `lo_exfil` (large-object import and export), `definer_escalation` (privilege escalation through chains of `SECURITY DEFINER` functions), and `ext_lateral` (extensions such as `dblink`, `file_fdw`, and `pg_stat_statements`). OOD-2 contains six families: `timing_oracle`, `xml_veil`, and `lockhog` are new technique families; `lo_stash` and `definer_backdoor` are new variants of the two techniques already used in OOD-1 (here called *reused*); and `benign_ops` is benign activity. Only three families are genuinely new, and the labels new, reused, and benign were assigned by family name and confirmed by the author.

How the data were split. The initial design applied `StratifiedGroupKFold` from scikit-learn [39] (three folds, shuffled, `random_state = 42`, groups by scenario, stratified by stage) to 17 scenarios, giving 11 training and 6 test scenarios (65 and 37 queries). The test fold was exposed during language-model debugging and was therefore merged into Dev, and the test set became the independent Holdout written by another agent. The resulting split (Dev 102 queries, 48%; Holdout 109 queries, 52%) is not a random split of one pool, there is no 70/30 ratio, and there is no validation set separate from Dev. What was kept: no scenario appears in two sets (a test checks identifiers and SQL), seeds are fixed, the Holdout was scored once, and the code was frozen before scoring.

3.2.5 Preprocessing and classification windows

Before classification every query is normalized by `preprocessing.py`: string literals become `'?'`, numeric literals become `?`, and the `aid` comment is removed. Table names are renamed to `t<n>_<h>`, where n counts names in order of first appearance and h is four characters of an MD5 hash of the seed and the name. One mapping is built for all statements of a run, it is deterministic (seed 42), and system objects (`pg_*`, `information_schema`, `public`) are kept. The mapping is therefore not a fresh random renaming per run or per session. The original statements are retained in the ground-truth files and the normalized text is the only classifier input.

A window consists of the target query and up to k previous queries of the same scenario ($k \in \{0, 2, 4\}$). Windows take their context from the true scenario identifier because the sessionizer could not group a single-address workload (Section 4.2.1); the context is therefore ideal and probably more favorable than in real use. A separate experiment on Dev corrupts the context in three ways to show how sensitive each method is: context taken from the sessionizer's own grouping (5-minute threshold), and mixes in which each context slot is replaced, with probability 0.5 or 1.0, by a query from another scenario. It applied the frozen Rule and Qwen at $k = 2$ and $k = 4$.

3.2.6 Methods under test

Rule-based classifier. The classifier is an ordered list of regular-expression rules applied to the target query (the SQL inside a `PREPARE` is unwrapped first), with the priority Exfiltration, Impact, Credential Access, Collection, Discovery, and Noise. For example, a mention

of `pg_shadow` or `pg_authid` gives Credential Access, `DROP TABLE`, role changes, `COPY ... FROM`, and `CREATE EXTENSION` give Impact, and `COPY ... TO` or `lo_export` give Exfiltration. The window context changes the label in only one case: a `DO` block with hidden content takes the stage (Discovery or Collection) of the earlier queries; otherwise the context only determines which earlier queries are listed as evidence (the queries that satisfy a stage-specific condition, plus the target). When no rule matches, the result is Noise. Rules were written and refined on Dev only. Because its label hardly depends on context, the Rule's score is almost independent of k by construction.

Language model. The model is Qwen3 8B served by Ollama [40] with no fine-tuning, seed 42, temperature 0, and the thinking mode off. Ollama version 0.40.11 ran locally on an Intel Core i7-1165G7 (4 cores, 8 threads) with 15.8 GB of memory and only an integrated Intel Iris Xe graphics unit, so inference ran on the CPU. The author confirmed that this is the experiment machine. The code does not store the software version in the result files. Candidate models were considered at the start (Qwen2.5-7B, Llama-3.1-8B, and Mistral-7B); Llama 3.1 8B was used first and then replaced by Qwen3 8B, and the other two were not tried.

The prompt is a plain few-shot prompt, not a multi-step guided one; the literature [4], [18], [25] motivated the explicit output constraints and the care taken with context. It was developed on Dev through an ablation of components A to F (Section 4.2.2 and Appendix B). The frozen version B+C+E contains a role and task statement, the six labels, the output format, a fixed list of few-shot examples (3 examples when $k = 0$ and 10 when $k \geq 2$, without retrieval), explicit `CONTEXT` and `TARGET` markers with an instruction to classify only the target, and stricter label definitions. Evidence always includes the target query: the code adds the target identifier if the model leaves it out. Because the examples that contain context are dropped at $k = 0$, a change from $k = 0$ to $k \geq 2$ changes both the context and the number of examples (3 versus 10); only the comparison between $k = 2$ and $k = 4$, which share the same 10 examples, isolates the length of the history. The verbatim prompt for a window with $k = 2$ is reproduced in Appendix G; a schematic follows.

```
You are a cybersecurity analyst. Classify only the TARGET query into
exactly one of: Discovery, Credential Access, Collection,
Exfiltration, Impact, Noise.
Return only JSON: {"label": "<label>", "evidence_ids": ["<id>", ...]}
```

```
No text outside the JSON.

[label definitions and fixed few-shot examples]

CONTEXT: [q1 ..., q2 ...]
TARGET:  [q3 ...]
```

The first `{...}` object in the reply is parsed with `json.loads` and then checked against the schema (a label outside the six allowed labels, or evidence that is not a list, is a schema error). Parse errors and schema errors are counted separately and neither is retried; generation is limited to 128 tokens. The replacement label for an invalid output is Noise, which can coincide with the true label; in the experiments there were no invalid outputs, so this choice did not affect any number. An evidence identifier is fabricated if it does not correspond to any query in the window; fabricated identifiers are counted and then dropped from the evidence set, and the fabricated-evidence rate is their number divided by all identifiers returned (including the target identifier that the code adds).

Combination strategies. Three strategies combine the Rule and Qwen labels. H1 accepts the label automatically when the two methods agree and sends the window to a human reviewer when they disagree. H3 follows the Rule except when Qwen says Credential Access: it hands to the language model a class that the rules tend to miss, namely the only class on which the model clearly beat the rules on Dev. H4 keeps the Rule label and uses the Qwen label only when the Rule returns Noise, that is, it hands to the language model the windows that the rules failed to recognise. H3 was selected on Dev; H4 was conceived after OOD-1 results were known. H4 was computed afterwards from the stored Rule and Qwen labels by a deterministic function and not by rerunning the model, and it is therefore marked † in the tables.

3.3 Sampling and Data Analysis Techniques

3.3.1 RQ1: coverage and session reconstruction

Action-level recall is computed for each label group g (stage or gap tag) and each configuration p:

$$\text{Recall}_{p,g} = \frac{\text{number of actions in } g \text{ with a matching log record under } p}{\text{number of actions in } g}.$$

The unit is the action: 136 actions consist of 102 SQL statements, 19 failed logins, and 15 connection events. Matching uses the `aid` tag, and by time, IP and user for events without SQL. Attribute completeness is the share of matched records that contain each attribute. Sessions are grouped by source IP and user, and a new session starts when the gap between consecutive events exceeds the threshold (1, 5, 15, or 30 minutes); the database is not part of the key. Quality is measured by pairwise precision, recall, and F1 over pairs of events. Exact matching of whole sessions was not computed.

Selection rule: a configuration is eligible if its recall is at least 95% overall and for every stage; among eligible configurations the one with the fewest log records is selected. This rule does not include a gate on the six attributes, because no configuration has all six in the strict sense. No interval is reported for RQ1, because the 136 actions come from 17 scenarios and are not independent.

3.3.2 RQ2: classification, evidence, and context

Classification performance is reported as macro-averaged F1, the unweighted mean of per-class F1 scores, together with per-class results and confusion matrices. The primary registered measure is macro-F1 at $k = 2$; $k = 0$ and $k = 4$ are exploratory and no "best k" was selected. Accuracy is a different measure and is used only for H1. Technical reliability is the invalid-output rate and the fabricated-evidence rate (fabricated identifiers divided by identifiers returned).

For a target query t, the ground-truth evidence set is $E_t^{\text{gt}} = \{t\} \cup \{d \in \text{depends_on}(t) : d \text{ lies in the window}\}$. The primary evidence measure is the Jaccard index $|E_t^{\text{pred}} \cap E_t^{\text{gt}}|/|E_t^{\text{pred}} \cup E_t^{\text{gt}}|$ averaged over windows, where two empty sets give 1; precision, recall, and F1 of the evidence sets are additional measures taken from each run's `results.json`. Evidence is evaluated only for $k \geq 2$, because at $k = 0$ the window contains only the target.

Ground-truth quality was checked after the rules and prompt were frozen. A seeded random sample of 20% of the $k = 4$ Dev windows that have context (20 windows, seed 42, 12 scenarios, not stratified) was reviewed by one reviewer, Claude, who is an AI and not a group member, blind to the automatic evidence. The reviewer's criterion (the queries needed to explain the label) is stricter than `depends_on`. The Jaccard index between the automatic and the reviewed sets estimates agreement between two ways of assigning evidence and does not prove that the ground truth is correct.

The effect of context compares $k \in \{0, 2, 4\}$ by paired differences in macro-F1 within each

method. H1 is described by four numbers: coverage (share of windows accepted), accuracy on accepted windows, accuracy of the Rule on all windows, and the accuracy of each method on the windows H1 sends to review. The difference between the accuracy on accepted windows and the Rule accuracy on all windows compares two different sets of windows (a selection effect) and is not the accuracy of the whole process, which was not measured.

3.3.3 RQ3: performance drop on unseen families

For each method the drop is

$$\Delta F_1 = F_1^{\text{Holdout}} - F_1^{\text{unseen}},$$

so that a positive value means a drop. It is computed on the classes that the two sets share: OOD-1 has five (no Noise class), so the Holdout score is recomputed on the same five classes and differs from the six-class RQ2 score; OOD-2 has the same six classes as the Holdout. The difference between two drops is $D = \Delta F_1(\text{A}) - \Delta F_1(\text{B})$; $D > 0$ means that B drops less than A. Here $D = \Delta F_1(\text{Rule}) - \Delta F_1(\text{Qwen})$, so $D > 0$ means Qwen drops less. A smaller drop does not mean a higher score on the unseen set. Because the sets differ in author, size, and class proportions, a drop is not caused by novelty alone.

Novelty of each family was scored from 1 to 3 on three factors (Table 3.4) by the person who built the family, once, before the unseen sets were scored. There was no second rater and no inter-rater agreement. The score for `ext_lateral` in OOD-1 is an estimate made from the description of the family; no original scoring existed, the author decided to keep it, and the report marks it as an estimate. With only three and six families the scores are used descriptively.

Table 3.4: Rubric for characterizing the novelty of a family. Dependency length counts the `depends_on` chain and is unrelated to k.

Factor	1	2	3
SQL structure	Only names, parameters, or conditions differ	Nested statements or unusual combinations of conditions	Multi-layer or second-order constructions, string-handling functions
PostgreSQL-specific functions	Standard SQL	Some system-information functions	Specialized functions or advanced extensions
Dependency chain	Independent steps or a chain of length 1	Chains of length 2–3	Long-range, fragmented chains with distant steps

3.3.4 Uncertainty and multiple comparisons

Confidence intervals are bootstrap percentile intervals over scenarios (scenarios are resampled, not queries, because queries of one scenario are dependent) with 1000 resamples and seed 42, paired between methods so that both are scored on the same resample. The Holdout and the unseen sets are resampled independently when a drop is computed. A supplementary script (`rq_extras`) used 10 000 resamples for H1, absolute differences, and evidence differences. A confidence interval that excludes zero is read as evidence of a difference; an interval that contains zero means that the evidence is insufficient, and does not mean that the two methods are equivalent. Unless stated otherwise, intervals are nominal and not adjusted for multiple comparisons.

Fixed class set. The default macro-F1 averages only the classes present in a resample. A rare class (Credential Access in OOD-1: one query in one of nine scenarios) was absent in 35.1% of resamples, and a few other classes in at most 5.3%. The main intervals for ΔF_1 and D therefore use an estimator with a fixed class set in which an absent class scores $\mathrm{F1} = 0$. This choice was made after the problem was discovered and was not registered in advance. The default estimator is kept as a sensitivity analysis (Appendix A). The point estimates of the two estimators are identical.

Six-comparison family. The six differences $D(\text{Rule} - \text{Qwen})$ (two unseen sets times $k \in \{0, 2, 4\}$) were adjusted together with a single-step max-$|t|$ procedure in the style of Westfall and Young. In each of 10 000 resamples (seed 42) the Holdout was drawn once for all six estimates and each unseen set once for the three values of k. With se_j the standard deviation of the resampled D_j^* and $t_j^* = (D_j^* - D_j)/se_j$, the critical value c is the 95th percentile of $\max_j |t_j^*|$ ($c = 2.592$ here, against 1.96 for a single comparison), the simultaneous interval is $D_j \pm c\, se_j$, and the adjusted p value is $(1 + \#\{\max_k |t_k^*| \geq |D_j/se_j|\})/(B + 1)$. The protocol registered the type of comparison (a confidence interval of the difference of two drops) but did not define this family of six or any multiplicity procedure; the family was defined at the final stage, before the adjustment was computed. All other comparisons (H1, H3, H4, subgroups, evidence, and the three values of k in RQ2) are nominal or exploratory.

3.4 Limitations of the Methodology

The limitations of the whole project are listed in Section 1.5; the following concern the method specifically.

- **Small, simulated data.** The ground truth reflects the template design, and the scenarios are informed by the literature [1], [5], [14] but are not recordings of real attacks. Per-class estimates rest on a few queries; some classes have one to six queries in a set.
- **Ideal context.** Windows use the true scenario identifier. Single-address simulation made real session grouping unusable (pairwise F1 about 0.11), and the corruption experiment was run on Dev only and not on the Holdout.
- **Prompt tuned on Dev.** Definition E was written from Dev errors, so Dev scores overstate performance. The fall of 0.018 in both methods from Dev to Holdout shows no large drop but is not evidence of the absence of leakage, and the optimism could not be measured.
- k **confounded with the number of examples.** At $k = 0$ the prompt shows 3 examples and at $k \geq 2$ it shows 10.
- **Intervals do not cover uncertainty between families.** The bootstrap treats scenarios as independent, but each unseen family has only three scenarios and only three families are genuinely new. Intervals may be narrower than the real uncertainty about other technique families.
- **Differences between data sets.** ΔF_1 compares sets that differ in author, size, class proportions (Impact is 36% of OOD-2) and difficulty. Credential Access has one query in OOD-1.
- **Post hoc choices.** The fixed-class estimator, the new/reused/benign subgroups, the comparisons involving H4, and the six-comparison family were defined after seeing results.
- **Single reviewer and thresholds.** Evidence review used one AI reviewer; the 95% criterion and the notion of sufficient attributes are the group's own choices; novelty scores were given once by the family author (and one is an estimate).
- **Freeze evidence.** The repository has only two commits, so the order of edits is documented by file modification times and the progress log: `rule_based.py` (26 October,

14:37), `llm_local.py` (26 October, 21:46, prompt B+C+E), `hybrid.py` (27 October, 08:03, H4 added after OOD-1), and `preprocessing.py` (27 October, 22:16, a condition on `session_logical`; inferred from the code to leave Holdout and unseen windows unaffected, not checked on the sealed data).

- **Statistical procedures without citation.** Cluster bootstrap, the max-$|t|$ adjustment, pairwise F1, and the session-gap thresholds are standard procedures, but no verified reference for them is cited in this report, and none was invented.
- **Non-determinism.** Temperature 0 and a fixed seed gave identical Holdout macro-F1 in a rerun, but results may differ on other hardware or software versions.
- **Not measured.** Inference CPU and memory, logging overhead (pgbench), query latency, and log volume in bytes (volume is counted in records).

Reproducibility and ethics

All generator code, parser code, prompts, rule definitions, seeds, and split lists are in a version-controlled repository, and the commands that regenerate each table are listed in Appendix F. The honeypot holds fabricated data only, is not exposed to external networks, and runs no real malware. The scenario generator produces SQL statements only, and no live attack infrastructure was used.

CHAPTER 4

EXPERIMENTAL AND RESULTS

4.1 Introduction

This chapter reports the experimental results in the order of the research questions: logging and session reconstruction (RQ1, Section 4.2.1), classification and evidence on known families (RQ2, Section 4.2.2), and performance on unseen families (RQ3, Section 4.2.3). Section 4.3 analyzes the intervals, Section 4.4 states what the results do and do not support, and the last two sections compare the results with the literature and give their implications. Every number comes from the final results table of the project (`final_results_tables.csv`), the saved outputs of the statistical scripts, or the development log, and the tables state the data set, the metric, the sample size, and the type of interval.

Three conventions apply throughout. (1) ΔF_1 is the score on the Holdout minus the score on the unseen set, so a positive value is a drop, and $D = \Delta F_1(\text{Rule}) - \Delta F_1(\text{Qwen})$ is positive when Qwen drops less. (2) Intervals are 95% bootstrap intervals over scenarios; an interval that contains zero means insufficient evidence and not equivalence. (3) Intervals are nominal and unadjusted unless the table says “simultaneous”. OOD-1 and OOD-2 denote the two unseen-family sets, “Qwen” denotes Qwen3 8B with the frozen prompt, and a dagger (†) marks H4 values that were computed after the fact. All data are simulated.

4.2 Presentation of Data

The four data sets are described in Table 3.3. RQ1 was run on Dev (136 actions of 17 scenarios), RQ2 on the Holdout (22 scenarios, 109 queries), and RQ3 on OOD-1 (9 scenarios, 57 queries) and OOD-2 (18 scenarios, 145 queries), compared with the Holdout.

4.2.1 RQ1: logging coverage and session reconstruction

RO1.1: missed actions. Table 4.1 gives recall per stage. Overall recall was 85.3% for C0 (116 of 136 actions), 89.0% for C1 (121 of 136), and 100% for C2 (136 of 136). The lowest recall by stage was Privilege Escalation (53.8% for C0 and C1), which consists of connection events without SQL, followed by Credential Access (69.2% for C0, 76.9% for C1). C2 recorded every action at every stage. On the unseen families, C2 recorded 62 of 63 actions in OOD-1 and 159 of 159 in OOD-2. Because C2 enables connection logging by design, its complete coverage of connection events follows from the configuration.

Table 4.1: RQ1: action-level recall by configuration and stage (Dev, 136 actions, 17 scenarios; no interval because the actions are not independent). Number of log records written under each configuration in the last row.

Stage	n	C0	C1	C2
All actions	136	85.3% (116/136)	89.0% (121/136)	100.0% (136/136)
Discovery	35	91.4% (32/35)	91.4% (32/35)	100.0% (35/35)
Credential Access	26	69.2% (18/26)	76.9% (20/26)	100.0% (26/26)
Privilege Escalation	13	53.8% (7/13)	53.8% (7/13)	100.0% (13/13)
Collection	18	83.3% (15/18)	100.0% (18/18)	100.0% (18/18)
Exfiltration	9	100.0% (9/9)	100.0% (9/9)	100.0% (9/9)
Impact	21	100.0% (21/21)	100.0% (21/21)	100.0% (21/21)
Noise	14	100.0% (14/14)	100.0% (14/14)	100.0% (14/14)
Log records written	–	199	204	322

Table 4.2 cuts recall by gap tag. C0 recorded only 54.5% of prepared-statement actions (6 of 11), and both C0 and C1 recorded only 36.8% of multi-connection actions (7 of 19). C2 reached 100% for every tag that was tested. The "log evasion" group was not tested. The earlier expectation that C0 and C1 would not log failed statements and failed logins was not borne out: all three configurations recorded them (19 of 19 failed logins and 11 of 11 error queries), because `log_min_error_statement` defaults to `error`.

RO1.2: attributes and sessions. Table 4.3 gives the presence of each attribute in the records matched to a scenario action. User, source IP, database, and result were present in every matched record under all three configurations. Statement text was present in 83.6% (C0), 84.3% (C1), and 75.0% (C2) of records, being absent for the failed-login events (19 in every configuration) and, in C2, the 15 connection events, which have no SQL, and bound parameters were present in none of the C0 and C1 records and in 3.7% (5 of 136) of the C2 records. The definition

Table 4.2: RQ1: recall by gap tag (Dev; an action can carry several tags; counts are actions).

Gap tag	n	C0	C1	C2
`error_query`	11	100.0% (11/11)	100.0% (11/11)	100.0% (11/11)
`auth_failure`	19	100.0% (19/19)	100.0% (19/19)	100.0% (19/19)
`nested_dynamic`	11	100.0% (11/11)	100.0% (11/11)	100.0% (11/11)
`prepared_statement`	11	54.5% (6/11)	100.0% (11/11)	100.0% (11/11)
`pg_specific`	16	100.0% (16/16)	100.0% (16/16)	100.0% (16/16)
`log_evasion`	0	not tested	not tested	not tested
`multi_connection`	19	36.8% (7/19)	36.8% (7/19)	100.0% (19/19)

of “sufficient attributes” was not confirmed, so no composite count such as “five of six” is reported.

Table 4.3: RQ1: share of matched log records that contain each session attribute (Dev).

Attribute	C0 (116 records)	C1 (121)	C2 (136)
user	100.0% (116/116)	100.0% (121/121)	100.0% (136/136)
source IP (`remote_host`)	100.0% (116/116)	100.0% (121/121)	100.0% (136/136)
database	100.0% (116/116)	100.0% (121/121)	100.0% (136/136)
statement text	83.6% (97/116)	84.3% (102/121)	75.0% (102/136)
result (`error_severity`)	100.0% (116/116)	100.0% (121/121)	100.0% (136/136)
bound parameters	0.0% (0/116)	0.0% (0/121)	3.7% (5/136)

Session grouping was weak (Table 4.4). Pairs were formed among the matched records, and two records count as truly together if they belong to the same scenario. Pairwise precision was about 0.06, recall 0.57 to 0.62, and F1 about 0.11 for all three configurations; recall is below 1 because the grouping key includes the user, so a scenario that changes user is split. The values did not change when the inactivity threshold was varied over 1, 5, 15, and 30 minutes, because every scenario ran from one source address, so that IP and user could not separate sessions. This is a limit of the experimental design and not evidence about session grouping in a honeypot with several source addresses; 100% action recall does not mean that sessions can be rebuilt.

Table 4.4: RQ1: pairwise session reconstruction (Dev). Values are identical at thresholds of 1, 5, 15, and 30 minutes (single simulated IP).

Config.	Pairwise precision	Pairwise recall	Pairwise F1
C0	0.0585	0.6197	0.1069
C1	0.0601	0.5986	0.1092
C2	0.0617	0.5683	0.1113

RO1.3: selected configuration. Only C2 met the criterion (recall of at least 95% overall and for every stage), so C2 was selected. C2 is the minimum configuration that meets the criterion; it is not the lightest, because it wrote 322 log records against 199 for C0 and 204 for C1 (about 1.6 times as many; the unit is records, not bytes). Query latency and overhead were not measured.

4.2.2 RQ2: classification and evidence on known families

Prompt development on Dev. With the first prompt, Qwen reached a macro-F1 of 0.515 at $k = 2$ on Dev, against 0.841 for the rule-based method, mainly because Noise was labeled Collection (12 of 14 Noise queries) and Impact was labeled Exfiltration (13 of 21). Adding balanced examples (B) raised the score to 0.639, and marking the context and the target and asking for a label for the target only (C) raised it to 0.641 (+0.126 [+0.036, +0.240] against the first prompt); asking for a reason before the label (D) did not help (0.513). Combining B and C gave 0.706 ($k = 2$) and 0.726 ($k = 4$), and adding stricter label definitions (E) gave 0.787 [0.722, 0.834] at $k = 2$. A version with further examples (BCEF) reached 0.811 but was not distinguishable from BCE (+0.024 [-0.018, +0.081]), so the simpler B+C+E was frozen. The definitions in E were written from Dev errors, so these scores are optimistic. The ablation table is in Appendix B.

RO2.1: accuracy on the Holdout. Table 4.5 gives macro-F1 on the Holdout, scored once. At $k = 2$, the registered primary setting, the Rule scored 0.823 and Qwen 0.769. From Dev to Holdout the score fell by 0.018 for both methods (Rule 0.841 to 0.823; Qwen 0.787 to 0.769). Qwen produced no JSON parse error, no schema error, and no fabricated evidence identifier at any k on any of the three evaluation sets. Inference took on average 4.6, 7.8, and 6.8 seconds per window at $k = 0, 2, 4$ for Qwen on the CPU (Holdout); the Rule is effectively instantaneous. CPU and memory use were not measured. A rerun of the Holdout gave identical macro-F1 values.

Table 4.5: RQ2: macro-F1 on the Holdout (22 scenarios, 109 queries, 6 classes; nominal 95% CI; difference paired).

k	Rule	Qwen	Qwen − Rule
0 (exploratory)	0.826 [0.762, 0.904]	0.732 [0.646, 0.852]	-0.094 [-0.151, -0.034]
2 (primary)	0.823 [0.756, 0.888]	0.769 [0.701, 0.827]	-0.053 [-0.131, $+0.016$]
4 (exploratory)	0.823 [0.756, 0.888]	0.769 [0.692, 0.826]	-0.053 [-0.133, $+0.021$]

Per-class results and confusion matrices are in Appendix C. At $k = 2$ the Rule's errors were

Collection labeled Discovery (4 of 15) and Noise labeled Discovery or Collection (3 each of 21); Qwen's errors were Impact labeled Exfiltration (5 of 27) and Collection labeled Impact (5 of 15) or Discovery (2). Combining the methods did not change the picture on known families: H3 scored 0.004 to 0.006 lower than the Rule with an interval that contains zero, and H4 minus Rule was -0.041 [-0.096, $+0.023$], $+0.012$ [-0.029, $+0.061$] and $+0.016$ [-0.033, $+0.077$] at $k = 0, 2, 4$ (Table A.4 in the appendix).

RO2.2: evidence. Table 4.6 gives the mean Jaccard index between predicted and ground-truth evidence. On the Holdout at $k = 2$ the Rule scored 0.821 and Qwen 0.754, a paired difference of -0.067 [-0.129, -0.015]; at $k = 4$ the difference was -0.010 [-0.074, $+0.044$]. The manual review of 20 Dev windows at $k = 4$ (12 scenarios, one AI reviewer) gave a Jaccard of 0.775 [0.667, 0.883] between the automatic and the reviewed evidence: 12 windows matched, 1 had a superfluous query, 4 a missing query, and 3 differed. This is a preliminary measure of agreement between two ways of assigning evidence on Dev; it does not show that the ground truth is correct.

Table 4.6: Evidence agreement: mean Jaccard index with ground-truth evidence derived from `depends_on` (evidence only for $k \geq 2$; 95% CI over scenarios, nominal; Holdout 22, OOD-1 9, OOD-2 18 scenarios).

Set	k	Rule	Qwen	Qwen − Rule
Holdout	2	0.821 [0.761, 0.881]	0.754 [0.697, 0.811]	−0.067 [−0.129, −0.015]
	4	0.777 [0.708, 0.845]	0.767 [0.720, 0.819]	−0.010 [−0.074, +0.044]
OOD-1	2	0.696 [0.622, 0.756]	0.751 [0.675, 0.824]	+0.056 [−0.014, +0.136]
	4	0.612 [0.550, 0.664]	0.708 [0.645, 0.774]	+0.095 [+0.021, +0.193]
OOD-2	2	0.662 [0.607, 0.725]	0.717 [0.660, 0.778]	+0.055 [+0.026, +0.085]
	4	0.645 [0.591, 0.707]	0.737 [0.680, 0.794]	+0.092 [+0.052, +0.141]

RO2.3: effect of k and of corrupted context. The Rule's score hardly changed with k (0.826, 0.823, 0.823), as expected because its label depends on context only for hidden `DO` blocks. Qwen's observed score was higher at $k = 2$ and $k = 4$ (0.769) than at $k = 0$ (0.732), but every paired difference contains zero (Table 4.7). The comparison of $k = 0$ with $k \geq 2$ also changes the number of examples (3 versus 10), and only $k = 4$ minus $k = 2$ isolates the history length; that difference was $+0.000$ [-0.031, $+0.038$]. On Dev, Qwen lost score when context was corrupted: at $k = 2$ the difference from ideal context was -0.058 [-0.106, -0.000] with the sessionizer's grouping and -0.071 [-0.132, -0.004] with fully mixed context, and at $k = 4$ it was -0.089 [-0.144, -0.012] and -0.048 [-0.105, $+0.016$]; the 50% mix gave -0.013 and

-0.026 (both with intervals containing zero). The Rule changed by at most 0.016. The size of the loss did not follow the share of mixed context, so the conditions are not ranked. This experiment was run on Dev only.

Table 4.7: RQ2: paired differences in macro-F1 between values of k on the Holdout (22 scenarios; nominal 95% CI).

Method	Comparison (later − earlier)	Difference	95% CI
Rule	k=2 minus k=0	−0.003	[−0.021, +0.004]
	k=4 minus k=2	±0.000	[±0.000, ±0.000]
	k=4 minus k=0	−0.003	[−0.021, +0.004]
Qwen	k=2 minus k=0	+0.037	[−0.061, +0.113]
	k=4 minus k=2	±0.000	[−0.031, +0.038]
	k=4 minus k=0	+0.037	[−0.049, +0.099]

H1: agreement as a filter. Table 4.8 gives the four numbers that describe H1. On the Holdout at $k = 2$, H1 accepted 77.1% of windows automatically and was right on 94.0% of them, while the Rule alone was right on about 82% of all windows. These are two different sets of windows (the accepted ones are the easier ones), so the difference of $+0.124$ is a selection effect and not the accuracy of the whole process, which was not measured. On the windows that H1 sent to a reviewer, the Rule was right on 40.0% and Qwen on 20.0%: these windows are hard for both methods.

4.2.3 RQ3: performance on unseen families

RO3.1: absolute scores and drops. Table 4.9 gives macro-F1 on each set with a fixed class set. On the Holdout restricted to the five classes of OOD-1 the Rule scored 0.872, 0.857, and 0.857 at $k = 0, 2, 4$ against 0.427, 0.418, and 0.419 on OOD-1; the difference is the drop in Table 4.10 and is not comparable with the six-class score of RQ2. All three methods dropped on both unseen sets, and all 18 intervals of ΔF_1 exclude zero (all values exceed the reporting threshold of 0.1, which was not a decision criterion). On OOD-1 the Rule dropped by 0.439 to 0.445, Qwen by 0.291 to 0.294, and H4† by 0.354 to 0.388; on OOD-2 the Rule dropped by 0.278 to 0.281, Qwen by 0.160 to 0.250, and H4† by 0.150 to 0.226.

Table 4.11 gives the difference between the drops of Rule and Qwen. On OOD-1 Qwen dropped less than the Rule at all three values of k ($D = +0.151$, $+0.145$, $+0.147$), and the simultaneous intervals, adjusted for the family of six comparisons, still exclude zero. On OOD-2 the result depends on k: $D = +0.121$ at $k = 0$ (the adjusted interval excludes zero by a

Table 4.8: H1 (accept when Rule and Qwen agree): coverage, accuracy on accepted windows, Rule accuracy on all windows, and each method's accuracy on the windows H1 sends to review. Accuracy is not macro-F1. Intervals for each value are in the project's results table; the interval is shown for the difference only.

Set	k	Coverage	Acc. accepted	Rule, all	H1 accepted − Rule all [CI]	Rule, review	Qwen, review
Holdout	0	0.817	0.854	0.817	+0.037 [−0.015, +0.098]	0.650	0.200
	2	0.771	0.941	0.817	+0.124 [+0.068, +0.189]	0.400	0.200
	4	0.761	0.940	0.817	+0.123 [+0.067, +0.189]	0.423	0.192
OOD-1	0	0.597	0.706	0.456	+0.250 [+0.190, +0.327]	0.087	0.435
	2	0.544	0.677	0.456	+0.221 [+0.134, +0.324]	0.192	0.423
	4	0.544	0.710	0.456	+0.254 [+0.175, +0.348]	0.154	0.423
OOD-2	0	0.600	0.483	0.379	+0.103 [+0.006, +0.234]	0.224	0.500
	2	0.655	0.432	0.379	+0.052 [−0.022, +0.155]	0.280	0.460
	4	0.579	0.500	0.379	+0.121 [+0.021, +0.273]	0.213	0.475

Table 4.9: Macro-F1 by data set with a fixed class set (95% CI over scenarios, nominal). H4† was computed afterwards from stored labels. OOD-1 has no Noise class, so the Holdout is also scored on the five shared classes.

Set (scenarios)	k	Rule	Qwen	H4†
Holdout (6 classes)	0	0.826 [0.749, 0.866]	0.732 [0.643, 0.789]	0.785 [0.700, 0.837]
	2	0.823 [0.736, 0.866]	0.769 [0.657, 0.823]	0.834 [0.732, 0.880]
	4	0.823 [0.736, 0.866]	0.769 [0.666, 0.823]	0.838 [0.740, 0.887]
Holdout on the 5 classes shared with OOD-1	0	0.872 [0.794, 0.923]	0.794 [0.683, 0.874]	0.857 [0.748, 0.932]
	2	0.857 [0.780, 0.910]	0.763 [0.683, 0.820]	0.848 [0.763, 0.903]
	4	0.857 [0.780, 0.910]	0.775 [0.684, 0.839]	0.861 [0.767, 0.922]
OOD-1 (5 classes)	0	0.427 [0.275, 0.574]	0.500 [0.347, 0.665]	0.503 [0.336, 0.671]
	2	0.418 [0.267, 0.568]	0.469 [0.326, 0.612]	0.483 [0.321, 0.636]
	4	0.419 [0.265, 0.569]	0.483 [0.341, 0.637]	0.472 [0.309, 0.632]
OOD-2 (6 classes)	0	0.545 [0.475, 0.601]	0.572 [0.473, 0.653]	0.635 [0.572, 0.687]
	2	0.545 [0.475, 0.601]	0.520 [0.404, 0.610]	0.609 [0.551, 0.662]
	4	0.545 [0.475, 0.601]	0.548 [0.444, 0.626]	0.626 [0.570, 0.682]

Table 4.10: Drop $\Delta F_1 = F_1(\text{Holdout}) - F_1(\text{unseen})$ on the shared classes; positive means a drop. Fixed-class estimator (main), 1000 resamples, nominal 95% CI. The default-estimator version is in Appendix A.

Unseen set	k	Rule	Qwen	H4†
OOD-1	0	+0.445 [+0.277, +0.605]	+0.294 [+0.104, +0.457]	+0.354 [+0.159, +0.526]
	2	+0.439 [+0.273, +0.599]	+0.294 [+0.131, +0.443]	+0.365 [+0.195, +0.529]
	4	+0.439 [+0.273, +0.600]	+0.291 [+0.115, +0.443]	+0.388 [+0.212, +0.559]
OOD-2	0	+0.281 [+0.182, +0.354]	+0.160 [+0.040, +0.272]	+0.150 [+0.055, +0.228]
	2	+0.278 [+0.172, +0.352]	+0.250 [+0.099, +0.362]	+0.226 [+0.110, +0.295]
	4	+0.278 [+0.172, +0.352]	+0.222 [+0.087, +0.330]	+0.213 [+0.100, +0.284]

margin of 0.001, with an adjusted p value of 0.0466), but $+0.028$ at $k = 2$ and $+0.056$ at $k = 4$, with intervals that contain zero.

Table 4.11: $D = \Delta F_1(\text{Rule}) - \Delta F_1(\text{Qwen})$; $D > 0$ means Qwen dropped less. Nominal 95% CI and simultaneous max-$|t|$ 95% CI for the family of six comparisons (10 000 resamples, seed 42, $c = 2.592$); p values unadjusted / adjusted. Fixed-class estimator. Comparisons with H4 are exploratory and are in Appendix A.

Unseen set	k	D	Nominal CI	Simultaneous CI	p unadj. / adj.	Reading of simultaneous CI
OOD-1	0	+0.151	[+0.090, +0.230]	[+0.058, +0.245]	0.0004 / 0.0008	excludes 0
	2	+0.145	[+0.066, +0.232]	[+0.037, +0.252]	0.0015 / 0.0059	excludes 0
	4	+0.147	[+0.059, +0.251]	[+0.022, +0.272]	0.0052 / 0.0176	excludes 0
OOD-2	0	+0.121	[+0.027, +0.207]	[+0.001, +0.240]	0.0096 / 0.0466	excludes 0 (borderline)
	2	+0.028	[−0.093, +0.139]	[−0.129, +0.185]	0.6502 / 0.9919	contains 0
	4	+0.056	[−0.073, +0.175]	[−0.107, +0.220]	0.3707 / 0.8568	contains 0

Absolute differences between the methods were computed separately (Appendix A, Table A.4). On OOD-1 Qwen minus Rule was $+0.073$ [$+0.031$, $+0.131$], $+0.051$ [$+0.016$, $+0.090$], and $+0.065$ [$+0.016$, $+0.121$] at $k = 0, 2, 4$. On OOD-2 as a whole it was $+0.027$ [-0.046, $+0.089$], -0.025 [-0.128, $+0.054$], and $+0.003$ [-0.100, $+0.094$], all containing zero, and H4† minus Rule was $+0.090$, $+0.064$, and $+0.081$, all excluding zero. Table 4.12 splits OOD-2 by technique type in a post hoc subgroup analysis with only 9, 6, and 3 scenarios. On the three genuinely new families, Qwen minus Rule was negative with intervals containing zero (about -0.08 to -0.02), while on the two families that reuse OOD-1 techniques it was $+0.136$ to $+0.169$ with intervals excluding zero. H4† minus Rule was positive on both the new

($+0.049$ to $+0.073$) and the reused families, but negative (-0.204 [-0.351, $+0.000$]) on the benign family, where the macro-F1 is the recall of Noise: the Rule recognised 12 of 27 Noise windows (44%) and H4† 7 of 27 (26%).

Table 4.12: OOD-2 split by technique type: absolute macro-F1 difference from the Rule (fixed class set; nominal 95% CI). Post hoc, exploratory subgroups with few scenarios.

Technique type (scenarios, classes)	k	Qwen − Rule	H4† − Rule
new technique (9 scenarios, 4 classes)	0	−0.075 [−0.198, +0.114]	+0.073 [+0.025, +0.125]
	2	−0.039 [−0.164, +0.122]	+0.049 [+0.002, +0.122]
	4	−0.023 [−0.143, +0.131]	+0.059 [+0.016, +0.145]
reused OOD-1 technique (6, 5 classes)	0	+0.149 [+0.121, +0.192]	+0.210 [+0.145, +0.341]
	2	+0.136 [+0.105, +0.179]	+0.214 [+0.153, +0.319]
	4	+0.169 [+0.104, +0.308]	+0.214 [+0.153, +0.319]
benign (3, 1 class)	0	−0.204 [−0.351, ±0.000]	−0.204 [−0.351, ±0.000]
	2	−0.204 [−0.351, ±0.000]	−0.204 [−0.351, ±0.000]
	4	−0.204 [−0.351, ±0.000]	−0.204 [−0.351, ±0.000]

Evidence on unseen families. Table 4.6 also gives the Jaccard values for the unseen sets. The Rule's mean Jaccard at $k = 2$ fell from 0.821 on the Holdout to 0.696 on OOD-1 and 0.662 on OOD-2, while Qwen's moved from 0.754 to 0.752 and 0.717 (differences of means, with no interval for the change). Qwen minus Rule was $+0.055$ [$+0.026$, $+0.085$] at $k = 2$ and $+0.092$ [$+0.052$, $+0.141$] at $k = 4$ on OOD-2 and $+0.095$ [$+0.021$, $+0.193$] at $k = 4$ on OOD-1, whereas on OOD-1 at $k = 2$ it was $+0.056$ [-0.014, $+0.136$]. On the Holdout at $k = 2$ the sign was reversed.

RO3.2: confusions and Noise. Table 4.13 counts the windows labeled Noise. OOD-1 has no true Noise, so every window labeled Noise is an error: the Rule labeled 18 to 20 of 57 windows Noise, Qwen 4 to 6, and H4† 2 to 3. On OOD-2 there are 27 true Noise windows; the Rule labeled 83 of 145 Noise (12 of them correct), Qwen 35 to 47, and H4† 34 to 46. In OOD-2, 42 of the 52 Impact windows were labeled Noise by the Rule at $k = 2$, and 31 by Qwen. The Rule defaults to Noise when no pattern matches, and Qwen labeled Discovery as Impact on OOD-1 (6 of 20 at $k = 2$). Qwen produced no fabricated evidence identifier and no invalid output on either set. Confusion matrices at $k = 2$ are in Appendix C.

Table 4.13: Windows labeled Noise by each method, with the number that were truly Noise in parentheses (counts of windows).

Set	k	Windows	True Noise	Rule	Qwen	H4†
Holdout	0	109	21	26 (14)	12 (7)	12 (7)
	2	109	21	22 (14)	14 (14)	13 (13)
	4	109	21	22 (14)	14 (13)	12 (12)
OOD-1	0	57	0	20 (0)	6 (0)	3 (0)
	2	57	0	18 (0)	5 (0)	2 (0)
	4	57	0	18 (0)	4 (0)	2 (0)
OOD-2	0	145	27	83 (12)	36 (7)	35 (7)
	2	145	27	83 (12)	47 (7)	46 (7)
	4	145	27	83 (12)	35 (7)	34 (7)

RO3.3: novelty scores. Table 4.14 lists the novelty scores 1 to 3 beside accuracy at $k = 2$. The scores were given once by the person who built each family before scoring, except for `ext_lateral`‡, whose scores are an estimate from the family description with no original scoring; the statement "scored before the run by the builder" is true for eight of the nine families. In OOD-1 the highest novelty score did not rank the hardest family: `definer_escalation` (3, 3, 3) had the highest Rule accuracy (0.688) of the three, while `ext_lateral` (2, 3, 2) had the lowest (0.208 for the Rule and 0.333 for Qwen). In OOD-2 the family with the lowest novelty on the dependency factor, `lockhog`, had the lowest accuracy (0.111 for the Rule, 0.185 for Qwen). With three and six families no correlation was computed and the table is descriptive; no factor can be named as the most influential.

Table 4.14: Novelty scores (SQL structure, PostgreSQL-specific functions, dependency chain; 1 to 3) and accuracy at $k = 2$ by family. ‡ estimated from the family description, not scored by the builder. n is the number of windows; one rater per family.

Set	Family	SQL	PG	Dep.	Technique	n	Rule	Qwen	H4†
OOD-1	`lo_exfil`	2	3	2	–	17	0.588	0.765	0.765
OOD-1	`definer_escalation`	3	3	3	–	16	0.688	0.688	0.812
OOD-1	`ext_lateral`‡	2	3	2	–	24	0.208	0.333	0.333
OOD-2	`timing_oracle`	2	2	3	new	22	0.455	0.318	0.500
OOD-2	`xml_veil`	3	3	2	new	27	0.333	0.519	0.519
OOD-2	`lockhog`	2	2	1	new	27	0.111	0.185	0.185
OOD-2	`lo_stash`	2	3	3	reused from OOD-1	21	0.286	0.571	0.714
OOD-2	`definer_backdoor`	2	3	3	reused from OOD-1	21	0.714	0.905	0.857
OOD-2	`benign_ops`	1	2	1	benign	27	0.444	0.259	0.259

H1 on unseen families. Table 4.8 shows that the accuracy of the windows that H1 accepted fell from about 0.94 on the Holdout to 0.68 to 0.71 on OOD-1 and 0.43 to 0.50 on OOD-2, so agreement between the two methods stopped being a sign of a correct label. On the windows H1 sent to review, Qwen was right more often than the Rule on both unseen sets (0.42 to 0.50 against 0.09 to 0.28).

4.3 Analysis of Results

Reading the intervals. The comparisons below follow one rule: only an interval that excludes zero counts as evidence. On the Holdout at $k = 0$ the difference Qwen $-$ Rule is -0.094 with an interval that excludes zero, so the Rule scored higher; this is an exploratory setting, and the three values of k are not adjusted for multiple comparisons. At the primary setting $k = 2$ the difference is -0.053 [-0.131, $+0.016$], and at $k = 4$ it is -0.053 [-0.133, $+0.021$]: the Rule has the higher point estimate, but the evidence is insufficient to claim a difference, and it does not show that the methods are equal. The paired differences between values of k all contain zero, with widths of about ± 0.1 for Qwen, so the data cannot say whether context helps Qwen, and they cannot say that it does not.

Drops on unseen families. A drop was found for every method and set (Table 4.10), and the sensitivity analysis with the default estimator gives the same point estimates, with lower bounds that are smaller for OOD-1 (for example the Rule at $k = 0$: [+0.196, +0.594] against [+0.277, +0.605]) and the same conclusions (Appendix A). The difference of drops D is more selective. After adjustment for the family of six comparisons the simultaneous intervals exclude zero for OOD-1 at every k and for OOD-2 at $k = 0$, and they contain zero for OOD-2 at $k = 2$ and $k = 4$. The OOD-2 cell at $k = 0$ is fragile: its lower bound is $+0.001$ and its adjusted p value is 0.0466, so a different bootstrap seed or a different choice of the family could change it. It is reported as borderline and is not used as primary evidence. OOD-1 has an independence problem, so its three cells are exploratory evidence as well. Excluding zero says only that the drop of Qwen was smaller on the sampled scenarios; it does not establish generalization to other technique families.

Why $D > 0$ is not a higher score. Part of D comes from Qwen being lower than the Rule on the Holdout. On the five shared classes at $k = 0$ the Holdout scores were 0.872 for the Rule

and 0.794 for Qwen, while on OOD-1 they were 0.427 and 0.500. On OOD-2 at $k = 2$ Qwen's absolute score (0.520) is below the Rule's (0.545), even though D is positive there. Qwen's lead on OOD-1 appears in absolute terms ($+0.051$ to $+0.073$), but not on OOD-2 as a whole.

Which technique types. The pooled OOD-2 result hides two different pictures (Table 4.12). On the three genuinely new families there is no evidence that Qwen exceeded the Rule. The advantage of Qwen concentrates on the two families that reuse techniques of OOD-1. Those techniques were also absent from Dev, the rules, and the prompt; why Qwen does better there is unknown, and the observation was made after seeing the results and was not a registered hypothesis. The standard errors reflect scenarios and not families, so intervals for this split are probably too narrow.

H4 and H1. H4† had higher macro-F1 than the Rule on OOD-2 as a whole (+0.064 to +0.090) and on its new families (+0.049 to +0.073, with the $k = 2$ lower bound at +0.002), did not change the result on the Holdout (intervals contain zero), and cost recognition of benign activity (Noise recall 44% to 26%). Its evidence is exploratory: H4 was conceived after OOD-1, and the subgroup has 9 scenarios. H1 filters out windows on which the methods disagree, but the accuracy of what it accepts fell to 0.43 to 0.50 on OOD-2, and the accuracy of the whole process including the human reviewer was not measured.

4.4 Interpretation of Results

Table 4.15 lists the main statements that the results do and do not support. The most important limit on interpretation is the one stated in every row: the data are simulated, small, and not fully isolated.

4.5 Comparison with Literature

The sources are described in this report only as far as their abstracts allow, and the comparison below is qualitative; the sources differ in domain and data, and no number is transferred from them.

Meng et al. argue that single-step LLM classification of security logs hallucinates technique names and loses context [4]. In this study the label space was closed and the evidence identifiers

Table 4.15: Main claims, the evidence for them, and the wording the evidence permits. The complete checklist is in Appendix D.

Claim	Evidence	Permitted reading
C2 records every action	136 of 136; OOD-1 62 of 63; OOD-2 159 of 159	True for the simulated workload tested; connection events are logged by design; one source address; evasion and overhead untested.
C2 is the lightest configuration	322 records against 199 and 204	False. C2 is the only configuration meeting the criterion and also the one with the most records.
Session grouping is good enough	Pairwise F1 about 0.11	Not supported: weak, a limit of the single-address design.
The Rule is better than Qwen on known families	-0.094 $[-0.151, -0.034]$ at $k = 0$; -0.053 $[-0.131, +0.016]$ at $k = 2$	Higher Rule score at $k = 0$ (exploratory); insufficient evidence at $k = 2$ and $k = 4$; not a statement of equivalence.
Context helps Qwen	$k = 2$ minus $k = 0$: $+0.037$ $[-0.061, +0.113]$	Not supported; $k = 0$ also differs in the number of examples.
Qwen drops less than the Rule on unseen techniques	D in Table 4.11	True on OOD-1 (all k) and on OOD-2 at $k = 0$ (borderline); insufficient at $k = 2, 4$; OOD-1 is not independent; $D > 0$ is not a higher score.
Qwen generalizes better to new techniques	New families: Qwen $-$ Rule -0.075 $[-0.198, +0.114]$ at $k = 0$	Not supported. Its advantage is on families reusing OOD-1 techniques (post hoc).
H4 is a general solution	OOD-2: $+0.064$ to $+0.090$; Holdout: intervals contain zero; benign recall 44% to 26%	Exploratory: higher than Rule on OOD-2 and its new families (nominal), at the cost of false alarms.
H1 makes the system more accurate	Accepted-window accuracy 0.94 against Rule 0.82 on the Holdout	Not supported as a statement about the whole system: different window sets; reviewer accuracy not measured.
Evidence ground truth was validated	Jaccard 0.775 [0.667, 0.883] on 20 Dev windows	Preliminary agreement with one AI reviewer; not a human validation.
ΔF_1 is caused by novelty	Positive drops on both sets	Not supported: the sets differ in author, size, and class mix.
There was no leakage from Dev to Holdout	Both scores fell by 0.018	Not supported: only "no large drop is seen".

were checked against the window; Qwen produced no invalid output and no fabricated identifier. This shows that this specific failure did not occur in these small windows, and it does not test the technique-name hallucination that RHINO describes. The sensitivity of Qwen to the prompt (a macro-F1 of 0.515 with the first prompt and 0.787 after development on Dev) agrees with reports that unguided prompting is unreliable and that instructions improve results [3], [25]. The fall of scores when context is corrupted is consistent with the concern of Meng et al. about decontextualized mappings, but our experiment did not isolate a mechanism.

Neves et al. found that a rule-based method was precise but had low recall on clinical phenotyping, that a zero-shot LLM had higher recall, and that combining the two can direct manual review to discordant cases [21]. The pattern on unseen techniques here is similar in direction: the Rule defaults to Noise when nothing matches and so missed attack windows (83 windows labeled Noise against 27 true in OOD-2), and the LLM recovered part of them; H1 and H4 are two ways to use the disagreement. On known families, in contrast, the Rule had the higher point estimate, whereas in the clinical study the LLM's recall was far higher, so the studies do not agree on which method is better in general. Mehavilla et al. report that LLMs did not outperform the best classical model on network logs [26], and Landauer et al. report that an LLM interpreted about one third of attack steps correctly and another third adequately [14]; neither result contradicts the finding here that Qwen did not clearly exceed a transparent baseline, but they concern other data.

Biswas and Niemi describe a trade-off between insufficient logging and the cost of redundant logging [10]. The result that only C2 met the recall criterion and also wrote the most records is consistent with this trade-off in log volume, but overhead on query processing was not measured here. Bridges et al. stress the shortage of realistic labeled data for honeypot research [38], which matches the dependence of this study on simulated data. No comparison is made between the 1 to 30 minute inactivity thresholds used here and any inactivity gaps reported by Song et al. [1], because the content of that source on this point was not verified for this report.

4.6 Implications of the Results

For a simulated workload of the kind used here, C2 was the only configuration that recorded all actions and met the criterion, and it was also the one that wrote the most records; the choice of C2 should be read as a minimum that meets a criterion chosen by the group, not as a configuration

suitable for every environment. Recording every action was not enough to rebuild sessions in a single-address setting, so a deployment would need several source addresses (or another session key) before windows with real context can be trusted.

Between the classifiers, on familiar behavior the rule-based method had the higher point estimate at every k and runs instantly, while Qwen needed 4 to 8 seconds per window and did not show a clear advantage; the rule-based method is therefore the reasonable default, with Qwen as an optional second opinion. For unfamiliar techniques the study shows a risk. Agreement between the two methods is a weak sign of correctness there (the accuracy of H1's accepted windows fell from 0.94 to 0.43 to 0.50), so an automatic gate based on agreement is unsafe without a human reviewer. H4 is a conditional proposal: it raised the score on OOD-2 but increased false alarms on benign activity, and its evidence is exploratory. None of these statements applies outside the simulated environment.

CHAPTER 5

DISCUSSION

5.1 Restate the Research Problem or Objectives

The project asked to what extent a high-interaction PostgreSQL honeypot combined with behavior analysis can record, identify, and explain attack stages from interaction logs, including when attackers use techniques that were not used to build the analysis. RQ1 asked which logging configurations miss which actions and which configuration suffices; RQ2 asked whether a rule-based method or a locally run few-shot language model assigns attack stages more accurately and selects better evidence; and RQ3 asked how much both methods lose on technique families withheld from development. The three form one chain: the configuration chosen in RQ1 supplied the logs for RQ2, and the methods frozen in RQ2 were applied unchanged in RQ3. All answers apply to simulated data.

5.2 Summarize Key Findings

F1. Complete action logs, weak session grouping. C2 recorded 136 of 136 simulated actions, against 116 and 121 for C0 and C1 (the gaps were prepared statements and multi-connection actions, not failed statements or logins). C2 was the only configuration that met the criterion and the one that wrote the most records. Sessions could not be rebuilt (pairwise F1 about 0.11), because the simulator used one source address.

F2. Known families: no distinguishable difference at the primary setting. The rule-based method had higher macro-F1 point estimates (0.823 against 0.769 at $k = 2$). At $k = 0$ the interval of the difference excludes zero, with the Rule higher (exploratory); at $k = 2$ and $k = 4$ it contains zero, so the evidence is insufficient, which is not evidence of equality.

F3. No evidence that context helps Qwen. Qwen's observed score was higher with context,

but all paired differences between values of k contain zero, and $k = 0$ also had fewer examples. On Dev, corrupted context lowered Qwen's score by up to 0.09 while the Rule hardly changed.

F4. Both methods lose a great deal on unseen techniques. The drop in macro-F1 was between about 0.15 and 0.45 for Rule, Qwen, and H4†, and every interval excludes zero.

F5. Qwen dropped less, but did not clearly beat the Rule on new techniques. Qwen's drop was smaller than the Rule's on OOD-1 at all k and on OOD-2 at $k = 0$ (the latter borderline after adjustment), not at $k = 2$ and $k = 4$ on OOD-2. This is a smaller drop and not a higher score, and on the three genuinely new families Qwen minus Rule was negative with intervals containing zero. Its advantage was on families that reuse OOD-1 techniques, a post hoc observation.

F6. Hybrid strategies trade false alarms for recall. H4†, which hands to the language model the windows the rules label Noise, raised the macro-F1 on OOD-2 and its new families (nominal) and left known families unchanged, but lowered Noise recall on benign activity from 44% to 26%. H1 stopped being a reliable filter on unseen techniques (accuracy of accepted windows 0.94, then 0.68 to 0.71, then 0.43 to 0.50).

These findings also have limits that the reader needs to keep in mind. The data are simulated and small; context is ideal; the test data were not fully isolated (Claude read part of the Holdout, the OOD-1 SQL was visible after the freeze, and OOD-2 was partly exposed through run-checker messages), and the effect of this was not measured; the prompt was tuned on Dev; intervals cover scenarios and not families, and only three families are genuinely new; ΔF_1 compares sets that differ in author, size, and class mix; several analyses were chosen after the results were seen; and evidence was checked by one AI reviewer. The findings therefore show trends in a controlled setting and cannot support a claim that either method generalizes better.

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 Conclusion

On a simulated PostgreSQL workload, the logging configuration with pgAudit and connection, error, and parameter logging (C2) recorded all 136 attack and benign actions, which no other tested configuration did, but it also wrote the most log records, and complete action-level recording did not allow sessions to be rebuilt: grouping by address and user reached a pairwise F1 of only about 0.11 because the simulator used one address. This weakness is a property of the experimental design and also the reason why the context given to the classifiers had to be tested for sensitivity.

On known technique families, the rule-based method scored higher than a locally run Qwen3 8B model (macro-F1 0.823 against 0.769 at the main setting of two preceding queries), but the difference could not be distinguished from zero at that setting, and the data could not show whether context helps the model. Rule, model, and hybrid strategies all lost between about 0.15 and 0.45 macro-F1 on technique families withheld from development. The model lost less than the rules on the first unseen set and in one setting of the second, but on the genuinely new families there was no evidence that it surpassed the rules, and a hybrid that defers to the model when the rules recognise nothing gained some score at the cost of more false alarms on benign activity.

These conclusions rest on small simulated data, ideal context, and test data that were not completely isolated from the people and tools that built the methods. They indicate trends and do not establish that any method generalizes better.

6.2 Future Work

- Use a simulator with several containers, each with its own source address, so that session reconstruction can be evaluated realistically, and measure the effect of imperfect sessions on classification (the corruption experiment was run on Dev only and should be repeated on the test data).
- Build a new, larger test set (at least 40 scenarios, with many Noise windows) written by people who do not take part in method development, with isolation that is enforced.
- Add a third set of unseen families with more families, because the uncertainty lies in the number of families and not in the number of scenarios per family.
- Fix the number of examples at each k so that context length can be separated from the number of examples.
- Add a second, human, evidence reviewer; pre-register the family of comparisons and the multiplicity procedure; and verify references for the statistical procedures that this report describes without citation.
- Complete what was not measured: evasion of logging, overhead with pgbench, and CPU and memory use during inference; reconsider the proxy or packet-capture configuration that was out of scope.

REFERENCES

[1] Y. Song, G. Smaragdakis, and H. Griffioen, "Decoy databases: Analyzing attacks on public facing databases," in *Proceedings of the 2025 ACM Internet Measurement Conference*, Madison WI USA: ACM, Oct. 28, 2025, pp. 709–726, ISBN: 979-8-4007-1860-1. DOI: 10.1145/3730567.3764481 Accessed: Sep. 25, 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/3730567.3764481

[2] M. Wegerer and S. Tjoa, "Defeating the database adversary using deception - a MySQL database honeypot," in *2016 International Conference on Software Security and Assurance (ICSSA)*, St. Pölten, Austria: IEEE, Aug. 2016, pp. 6–10, ISBN: 978-1-5090-4388-0. DOI: 10.1109/ICSSA.2016.8 Accessed: Sep. 25, 2026. [Online]. Available: http://ieeexplore.ieee.org/document/7861643/

[3] A. Schärmer, M. Landauer, F. Skopik, M. Wurzenberger, and M. Squarcina, "Let the alerts speak: LLM-based IDS alert interpretation for SOC triage," in *Availability, Reliability and Security. ARES 2026 International Workshops*, ser. Lecture Notes in Computer Science, Z. Afzal, P. Kochberger, B. Atli, and M. Asplund, Eds., vol. 16901, Cham: Springer Nature Switzerland, 2027, pp. 346–364, ISBN: 978-3-032-35578-2 978-3-032-35579-9. DOI: 10.1007/978-3-032-35579-9_19 Accessed: Oct. 27, 2026. [Online]. Available: https://link.springer.com/10.1007/978-3-032-35579-9_19

[4] F. Meng, J. Gui, Y. Li, and Y. Wu, *RHINO: Guided reasoning for mapping network logs to adversarial tactics and techniques with large language models*, Oct. 16, 2025. DOI: 10.48550/arXiv.2510.14233 arXiv: 2510.14233[cs.CR]. Accessed: Oct. 27, 2026. [Online]. Available: http://arxiv.org/abs/2510.14233

[5] J. Gómez-López, D. Penco-Hernández, Ó. D. Gómez-López, F. J. Martínez-López, and N. Padilla-Soriano, "A multi-agent framework for SQL injection auditing with LLM-driven post-exploitation and privilege escalation," *Computers*, vol. 15, no. 10, p. 658,

Sep. 29, 2026, ISSN: 2073-431X. DOI: 10.3390/computers15100658 Accessed: Oct. 27, 2026. [Online]. Available: https://www.mdpi.com/2073-431X/15/10/658

[6] A. Rocha and F. S. Pereira, "What is the most targeted database? using honeypots to characterize attacks on relational databases," *Procedia Computer Science*, vol. 256, pp. 114–121, 2025, ISSN: 18770509. DOI: 10.1016/j.procs.2025.02.103 Accessed: Sep. 25, 2026. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S1877050925004600

[7] J. Al Hafidz, C. Lim, and K. E. Silaen, "Enhancing detection postgresql shell honeypot with interactive query response," in *2025 IEEE 2nd International Conference on Cryptography, Informatics, and Cybersecurity (ICoCICs)*, Depok, Indonesia: IEEE, Oct. 22, 2025, pp. 255–260, ISBN: 979-8-3315-5481-1. DOI: 10.1109/ICoCICs68032.2025.11384049 Accessed: Oct. 27, 2026. [Online]. Available: https://ieeexplore.ieee.org/document/11384049/

[8] A. A. Karo Karo, C. Lim, and K. E. Silaen, "Capturing post-login behaviors in MS SQL honeypots: Profiling attackers via a protocol-aware TDS relay," in *2025 15th International Conference on Information & Communication Technology and System (ICTS)*, Denpasar, Indonesia: IEEE, Nov. 12, 2025, pp. 1–6, ISBN: 979-8-3315-6699-9. DOI: 10.1109/ICTS67612.2025.11369480 Accessed: Oct. 27, 2026. [Online]. Available: https://ieeexplore.ieee.org/document/11369480/

[9] M. Zimniak, J. R. Getta, and W. Benn, "Predicting database workloads through mining periodic patterns in database audit trails," *Vietnam Journal of Computer Science*, vol. 2, no. 4, pp. 201–211, Nov. 2015, ISSN: 2196-8888, 2196-8896. DOI: 10.1007/s40595-015-0042-0 Accessed: Sep. 25, 2026. [Online]. Available: http://link.springer.com/10.1007/s40595-015-0042-0

[10] D. Biswas and V. Niemi, "Transforming privacy policies to auditing specifications," in *2011 IEEE 13th International Symposium on High-Assurance Systems Engineering*, Boca Raton, FL, USA: IEEE, Nov. 2011, pp. 368–375, ISBN: 978-1-4673-0107-7 978-0-7695-4615-5. DOI: 10.1109/HASE.2011.51 Accessed: Sep. 25, 2026. [Online]. Available: http://ieeexplore.ieee.org/document/6113921/

[11] L. Zeng, Y. Xiao, and H. Chen, "Auditing overhead, auditing adaptation, and benchmark evaluation in linux," *Security and Communication Networks*, vol. 8, no. 18, pp. 3523–

3534, Dec. 2015, ISSN: 1939-0114, 1939-0122. DOI: 10.1002/sec.1277 Accessed: Sep. 25, 2026. [Online]. Available: https://onlinelibrary.wiley.com/doi/10.1002/sec.1277

[12] H. Choi and S. Lee, "Forensic analysis of SQL server transaction log in unallocated area of file system," *Forensic Science International: Digital Investigation*, vol. 46, p. 301 605, Oct. 2023, ISSN: 26662817. DOI: 10.1016/j.fsidi.2023.301605 Accessed: Sep. 25, 2026. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S2666281723001178

[13] K. Goseva-Popstojanova, G. Anastasovski, A. Dimitrijevikj, R. Pantev, and B. Miller, "Characterization and classification of malicious web traffic," *Computers & Security*, vol. 42, pp. 92–115, May 2014, ISSN: 01674048. DOI: 10.1016/j.cose.2014.01.006 Accessed: Sep. 25, 2026. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S0167404814000170

[14] M. Landauer, W. Hotwagner, T. Boenke, F. Skopik, and M. Wurzenberger, *CAM-LDS: Cyber attack manifestations for automatic interpretation of system logs and security alerts*, Mar. 4, 2026. DOI: 10.48550/arXiv.2603.04186 arXiv: 2603.04186[cs.CR]. Accessed: Oct. 27, 2026. [Online]. Available: http://arxiv.org/abs/2603.04186

[15] N. Hossain, "SLEUTH: Real-time attack scenario reconstruction from COTS audit data,"

[16] T. Zhu□, M. Li□, and Z. Wang□□, "TraceBack: Sequence-enhanced and causality-driven attack scenario reconstruction from provenance graphs,"

[17] M. Zambianco, C. Facchinetti, and D. Siracusa, "A proactive decoy selection scheme for cyber deception using MITRE ATT&CK," *Computers & Security*, vol. 148, p. 104 144, Jan. 2025, ISSN: 01674048. DOI: 10.1016/j.cose.2024.104144 arXiv: 2404.12783[cs.CR]. Accessed: Sep. 25, 2026. [Online]. Available: http://arxiv.org/abs/2404.12783

[18] Y. Meng et al., *Benchmarking LLM-assisted blue teaming via standardized threat hunting*, May 28, 2026. DOI: 10.48550/arXiv.2509.23571 arXiv: 2509.23571[cs.CR]. Accessed: Oct. 27, 2026. [Online]. Available: http://arxiv.org/abs/2509.23571

[19] L. Yang, A. Ye, Y. Liu, W. Lu, and C. Huang, "APTREC: APT tactic/technique recognition based on large language model," *Computer Networks*, vol. 280, p. 112 146, 2026,

ISSN: 1389-1286. DOI: https://doi.org/10.1016/j.comnet.2026.112146 [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1389128626001581

[20] Z. Wang, Z. Fan, Y. Li, X. Zhang, P. Dai, and Z. Tian, "Toward zero-shot multi-label identification of attack techniques: Enhancing LLM reasoning with semantic evidence discovery," *IEEE Transactions on Knowledge and Data Engineering*, vol. 38, no. 7, pp. 4791–4804, Jul. 2026, ISSN: 1041-4347, 1558-2191, 2326-3865. DOI: 10.1109/TKDE.2026.3682468 Accessed: Oct. 27, 2026. [Online]. Available: https://ieeexplore.ieee.org/document/11478373/

[21] B. Neves et al., "Zero-shot learning for clinical phenotyping: Comparing LLMs and rule-based methods," *Computers in Biology and Medicine*, vol. 192, p. 110 181, Jun. 2025, ISSN: 00104825. DOI: 10.1016/j.compbiomed.2025.110181 Accessed: Oct. 27, 2026. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S0010482525005323

[22] R. Duan, D. Li, Q. Tong, T. Yang, X. Liu, and X. Liu, "A survey of few-shot learning: An effective method for intrusion detection," *Security and Communication Networks*, vol. 2021, Y. Guo, Ed., pp. 1–10, Oct. 31, 2021, ISSN: 1939-0122, 1939-0114. DOI: 10.1155/2021/4259629 Accessed: Sep. 25, 2026. [Online]. Available: https://www.hindawi.com/journals/scn/2021/4259629/

[23] H. Liu and B. Lang, "Machine learning and deep learning methods for intrusion detection systems: A survey," *Applied Sciences*, vol. 9, no. 20, p. 4396, Oct. 17, 2019, ISSN: 2076-3417. DOI: 10.3390/app9204396 Accessed: Sep. 25, 2026. [Online]. Available: https://www.mdpi.com/2076-3417/9/20/4396

[24] A. Alharthi, M. Alaryani, and S. Kaddoura, "A comparative study of machine learning and deep learning models in binary and multiclass classification for intrusion detection systems," *Array*, vol. 26, p. 100 406, Jul. 2025, ISSN: 25900056. DOI: 10.1016/j.array.2025.100406 Accessed: Sep. 25, 2026. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S2590005625000335

[25] M. A. Rehman, S. I. A. Shah, A. Anwar, and N. Islam, *From flows to words: Can zero-/few-shot LLMs detect network intrusions? a grammar-constrained, calibrated evaluation on UNSW-NB15*, Oct. 26, 2025. DOI: 10.48550/arXiv.2510.17883 arXiv:

2510.17883[cs.CR]. Accessed: Sep. 25, 2026. [Online]. Available: http://arxiv.org/abs/2510.17883

[26] L. Mehavilla, M. Rodríguez, J. García, and Á. Alesanco, "Evaluating large language models effectiveness for flow-based intrusion detection: A comparative study with ML and DL baselines," *Artificial Intelligence Review*, vol. 59, no. 2, p. 50, Jan. 9, 2026, ISSN: 1573-7462. DOI: 10.1007/s10462-025-11432-2 Accessed: Sep. 25, 2026. [Online]. Available: https://link.springer.com/10.1007/s10462-025-11432-2

[27] J. Lust and A. P. Condurache, "A survey on assessing the generalization area of neural networks: Uncertainty, out-of-distribution and adversarial samples," *Neurocomputing*, vol. 706, p. 135 003, 2026, ISSN: 0925-2312. DOI: https://doi.org/10.1016/j.neucom.2026.135003 [Online]. Available: https://www.sciencedirect.com/science/article/pii/S092523122602401X

[28] Y. Zhou and Y. Wang, "GROD: Enhancing generalization of transformer with out-of-distribution detection," 2024.

[29] S. Guo, J. Wildberger, and B. Schölkopf, "OUT-OF-VARIABLE GENERALIZATION FOR DISCRIMINATIVE MODELS," 2024.

[30] V. Das and B. B. Nair, "A novel multi-stage attack dataset for smart home intrusion detection," *Data in Brief*, vol. 66, p. 112 770, Jun. 2026, ISSN: 23523409. DOI: 10.1016/j.dib.2026.112770 Accessed: Sep. 25, 2026. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S2352340926003239

[31] T. Noelia Nunez Migliorisi, "EMPIRICAL TELEMETRY-BASED METRICS FOR EVALUATING HONEYPOT REALISM AND DECEPTION EFFECTIVENESS," *International Journal of Network Security & Its Applications*, vol. 18, no. 1, pp. 01–12, Jan. 28, 2026, ISSN: 09752307. DOI: 10.5121/ijnsa.2026.18101 Accessed: Sep. 25, 2026. [Online]. Available: https://aircconline.com/ijnsa/V18N1/18126ijnsa01.pdf

[32] K. J. Ferguson-Walter and M. M. Major, "Examining the efficacy of decoy-based and psychological cyber deception,"

[33] A. I. Weinberg, *PHANTOM: Polymorphic honeytoken adaptation with narrative-tailored organisational mimicry*, May 4, 2026. DOI: 10.48550/arXiv.2605.02992 arXiv:

2605.02992[cs.CR]. Accessed: Sep. 25, 2026. [Online]. Available: http://arxiv.org/abs/2605.02992

[34] M. Msaad, S. Srinivasa, M. M. Andersen, D. H. Audran, C. U. Orji, and E. Vasilomanolakis, "Honeysweeper: Towards stealthy honeytoken fingerprinting techniques," in *Secure IT Systems*, H. P. Reiser and M. Kyas, Eds., vol. 13700, Series Title: Lecture Notes in Computer Science, Cham: Springer International Publishing, 2022, pp. 101–119, ISBN: 978-3-031-22294-8 978-3-031-22295-5. DOI: 10.1007/978-3-031-22295-5_6 Accessed: Sep. 25, 2026. [Online]. Available: https://link.springer.com/10.1007/978-3-031-22295-5_6

[35] M. Kouremetis et al., "Mirage: Cyber deception against autonomous cyber attacks in emulation and simulation," *Annals of Telecommunications*, vol. 79, no. 11, pp. 803–817, Dec. 2024, ISSN: 0003-4347, 1958-9395. DOI: 10.1007/s12243-024-01018-4 Accessed: Sep. 25, 2026. [Online]. Available: https://link.springer.com/10.1007/s12243-024-01018-4

[36] U. Wushishi, A. Hussain, M. I. Khalid, N. Hussain, M. Jamjoom, and Z. Ullah, "D3o-IIoT: Deep reinforcement learning-driven dynamic deception orchestration for industrial IoT security," *Scientific Reports*, vol. 16, no. 1, p. 2389, Dec. 21, 2025, ISSN: 2045-2322. DOI: 10.1038/s41598-025-33426-4 Accessed: Sep. 25, 2026. [Online]. Available: https://www.nature.com/articles/s41598-025-33426-4

[37] A. Safargalieva, A. Rüffer, and E. Vasilomanolakis, "OHRA: Dynamic multi-protocol LLM-based cyber deception,"

[38] R. A. Bridges, T. R. Mitchell, M. Muñoz, and T. Henriksson, *SoK: Honeypots & LLMs, more than the sum of their parts?* Apr. 7, 2026. DOI: 10.48550/arXiv.2510.25939 arXiv: 2510.25939[cs.CR]. Accessed: Oct. 27, 2026. [Online]. Available: http://arxiv.org/abs/2510.25939

[39] scikit-learn developers. "Stratifiedgroupkfold," Accessed: Oct. 28, 2026. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.StratifiedGroupKFold.html

[40] Ollama. "Ollama documentation," Accessed: Oct. 28, 2026. [Online]. Available: https://docs.ollama.com/

CHAPTER A

SUPPLEMENTARY RESULT TABLES

All tables are generated from `final_results_tables.csv`. Intervals are nominal 95% bootstrap intervals over scenarios unless stated.

Table A.1: Sensitivity: ΔF_1 with the default estimator (macro-F1 averaged over the classes present in each resample). Point estimates equal those of Table 4.10.

Unseen set	k	Rule	Qwen	H4†
OOD-1	0	+0.445 [+0.196, +0.594]	+0.294 [+0.016, +0.448]	+0.354 [+0.070, +0.516]
	2	+0.439 [+0.184, +0.588]	+0.294 [+0.023, +0.434]	+0.365 [+0.083, +0.513]
	4	+0.439 [+0.184, +0.589]	+0.291 [+0.026, +0.426]	+0.388 [+0.102, +0.544]
OOD-2	0	+0.281 [+0.168, +0.376]	+0.160 [+0.025, +0.296]	+0.150 [+0.025, +0.281]
	2	+0.278 [+0.166, +0.364]	+0.250 [+0.111, +0.370]	+0.226 [+0.097, +0.301]
	4	+0.278 [+0.166, +0.364]	+0.222 [+0.089, +0.335]	+0.213 [+0.087, +0.297]

Table A.2: Differences of drops, all pairs, fixed-class estimator (nominal CI). Rule − Qwen is the registered comparison; those with H4† are exploratory.

Unseen set	Pair	k	D [CI]
OOD-1	Rule − Qwen	0	+0.151 [+0.090, +0.230]
		2	+0.145 [+0.066, +0.232]
		4	+0.147 [+0.059, +0.251]
	Rule − H4	0	+0.091 [+0.024, +0.174]
		2	+0.073 [+0.024, +0.132]
		4	+0.050 [−0.002, +0.115]
	H4 − Qwen	0	+0.060 [+0.024, +0.094]
		2	+0.071 [+0.002, +0.148]
		4	+0.097 [+0.027, +0.176]
OOD-2	Rule − Qwen	0	+0.121 [+0.027, +0.207]
		2	+0.028 [−0.093, +0.139]
		4	+0.056 [−0.073, +0.175]
	Rule − H4	0	+0.131 [+0.051, +0.214]
		2	+0.052 [−0.011, +0.120]
		4	+0.065 [−0.013, +0.143]
	H4 − Qwen	0	−0.010 [−0.076, +0.047]
		2	−0.024 [−0.119, +0.066]
		4	−0.009 [−0.099, +0.071]

Table A.3: Sensitivity: differences of drops with the default estimator.

Unseen set	Pair	k	D [CI]
OOD-1	Rule − Qwen	0	+0.151 [+0.092, +0.243]
		2	+0.145 [+0.066, +0.238]
		4	+0.147 [+0.060, +0.260]
	Rule − H4	0	+0.091 [+0.029, +0.187]
		2	+0.073 [+0.025, +0.149]
		4	+0.050 [−0.002, +0.128]
	H4 − Qwen	0	+0.060 [+0.023, +0.096]
		2	+0.071 [±0.000, +0.148]
		4	+0.097 [+0.022, +0.178]
OOD-2	Rule − Qwen	0	+0.121 [+0.027, +0.207]
		2	+0.028 [−0.093, +0.141]
		4	+0.056 [−0.073, +0.176]
	Rule − H4	0	+0.131 [+0.051, +0.215]
		2	+0.052 [−0.011, +0.122]
		4	+0.065 [−0.013, +0.143]
	H4 − Qwen	0	−0.010 [−0.076, +0.049]
		2	−0.024 [−0.121, +0.068]
		4	−0.009 [−0.102, +0.072]

Table A.4: Absolute macro-F1 differences from the Rule, fixed-class estimator (nominal CI; scenarios in parentheses). H4† was computed after the fact.

Set	k	Qwen − Rule	H4† − Rule
Holdout (22)	0	-0.094 [-0.148, -0.031]	-0.041 [-0.096, $+0.023$]
	2	-0.053 [-0.125, $+0.019$]	$+0.011$ [-0.029, $+0.061$]
	4	-0.053 [-0.129, $+0.023$]	$+0.016$ [-0.033, $+0.077$]
OOD-1 (9)	0	$+0.073$ [$+0.031$, $+0.131$]	$+0.076$ [$+0.026$, $+0.140$]
	2	$+0.051$ [$+0.016$, $+0.089$]	$+0.065$ [$+0.023$, $+0.103$]
	4	$+0.065$ [$+0.016$, $+0.121$]	$+0.054$ [$+0.014$, $+0.094$]
OOD-2 all (18)	0	$+0.027$ [-0.045, $+0.089$]	$+0.090$ [$+0.044$, $+0.145$]
	2	-0.025 [-0.128, $+0.054$]	$+0.064$ [$+0.018$, $+0.120$]
	4	$+0.003$ [-0.100, $+0.093$]	$+0.081$ [$+0.034$, $+0.139$]

CHAPTER B

DEVELOPMENT-SET RESULTS

These results come from the Dev set (102 queries, 17 scenarios), on which prompts were tuned; scores are optimistic. Values are transcribed from the project log.

Table B.1: Prompt ablation for Qwen3 8B on Dev (macro-F1). A: same examples at every k; B: balanced examples added; C: CONTEXT/TARGET markers and target-only instruction; D: reason before label; E: stricter label definitions; F: extra Collection examples. The first prompt is the starting point; the rule-based method is the reference.

Prompt	$k=0$	$k=2$	$k=4$	Note
First prompt	0.698	0.515	0.472	Noise labeled Collection; Impact labeled Exfiltration
A	0.631	–	–	worse than first prompt at $k=0$
B	–	0.639	–	no distinguishable gain alone
C	0.670	0.641	–	$+0.126$ [$+0.036$, $+0.240$] at $k=2$
D	0.622	0.513	–	no effect
B+C	–	0.706	0.726	
B+C+E (frozen)	–	0.787	–	0.787 [0.722, 0.834]
B+C+F	–	0.743	–	
B+C+E+F	–	0.811	0.822	not distinguishable from B+C+E
Rule (reference)	0.800	0.841	0.841	

Table B.2: Sensitivity of Qwen to corrupted context on Dev (macro-F1 and difference from ideal context; 95% CI). The "sessionizer" condition has only two mixed sessions.

Condition	Score	Difference	Interval
$k=2$ ideal	0.787	–	–
$k=2$ sessionizer	0.729	-0.058	[-0.106, -0.000]
$k=2$ mix 50%	0.774	-0.013	contains 0
$k=2$ mix 100%	0.716	-0.071	[-0.132, -0.004]
$k=4$ ideal	0.782	–	–
$k=4$ sessionizer	0.693	-0.089	[-0.144, -0.012]
$k=4$ mix 50%	0.756	-0.026	contains 0
$k=4$ mix 100%	0.735	-0.048	[-0.105, $+0.016$]

CHAPTER C

CONFUSION MATRICES AT $k = 2$

Rows are true labels and columns are predicted labels; "other" is any label outside the six classes (none occurred). Matrices for $k = 0$ and $k = 4$ are in `report_extra_tables.md`. Disc. = Discovery, Cred. = Credential Access, Coll. = Collection, Exfil. = Exfiltration.

Table C.1: Confusion matrix, Holdout, Rule, $k = 2$.

True \ predicted	Disc.	Cred.	Coll.	Exfil.	Impact	Noise	other
Discovery	18	0	0	0	0	3	0
Credential Access	0	11	0	0	0	2	0
Collection	4	0	9	0	0	2	0
Exfiltration	0	0	0	12	0	0	0
Impact	0	0	1	0	25	1	0
Noise	3	0	3	0	1	14	0

Table C.2: Confusion matrix, Holdout, Qwen, $k = 2$.

True \ predicted	Disc.	Cred.	Coll.	Exfil.	Impact	Noise	other
Discovery	18	1	0	0	2	0	0
Credential Access	0	11	0	0	2	0	0
Collection	2	1	7	0	5	0	0
Exfiltration	0	0	0	12	0	0	0
Impact	0	0	0	5	22	0	0
Noise	4	0	1	0	2	14	0

Table C.3: Confusion matrix, Holdout, H4†, $k = 2$.

True \ predicted	Disc.	Cred.	Coll.	Exfil.	Impact	Noise	other
Discovery	19	1	0	0	1	0	0
Credential Access	0	11	0	0	2	0	0
Collection	4	1	10	0	0	0	0
Exfiltration	0	0	0	12	0	0	0
Impact	0	0	1	0	26	0	0
Noise	3	0	3	0	2	13	0

Table C.4: Confusion matrix, OOD-1, Rule, $k = 2$.

True \ predicted	Disc.	Cred.	Coll.	Exfil.	Impact	Noise	other
Discovery	10	0	0	0	5	5	0
Credential Access	0	0	1	0	0	0	0
Collection	0	0	9	0	0	6	0
Exfiltration	0	0	3	5	1	2	0
Impact	1	0	2	0	2	5	0
Noise	0	0	0	0	0	0	0

Table C.5: Confusion matrix, OOD-1, Qwen, $k = 2$.

True \ predicted	Disc.	Cred.	Coll.	Exfil.	Impact	Noise	other
Discovery	12	1	1	0	6	0	0
Credential Access	0	0	0	0	0	1	0
Collection	4	0	7	0	1	3	0
Exfiltration	2	0	3	4	2	0	0
Impact	0	0	0	0	9	1	0
Noise	0	0	0	0	0	0	0

Table C.6: Confusion matrix, OOD-1, H4†, $k = 2$.

True \ predicted	Disc.	Cred.	Coll.	Exfil.	Impact	Noise	other
Discovery	13	0	1	0	6	0	0
Credential Access	0	0	1	0	0	0	0
Collection	3	0	10	0	1	1	0
Exfiltration	1	0	4	5	1	0	0
Impact	1	0	2	0	6	1	0
Noise	0	0	0	0	0	0	0

Table C.7: Confusion matrix, OOD-2, Rule, $k = 2$.

True \ predicted	Disc.	Cred.	Coll.	Exfil.	Impact	Noise	other
Discovery	12	0	0	0	0	9	0
Credential Access	0	6	0	0	0	0	0
Collection	0	0	10	0	0	17	0
Exfiltration	0	0	0	9	0	3	0
Impact	1	0	3	0	6	42	0
Noise	14	0	1	0	0	12	0

Table C.8: Confusion matrix, OOD-2, Qwen, $k = 2$.

True \ predicted	Disc.	Cred.	Coll.	Exfil.	Impact	Noise	other
Discovery	17	0	0	0	0	4	0
Credential Access	0	6	0	0	0	0	0
Collection	5	2	9	2	6	3	0
Exfiltration	1	0	2	6	1	2	0
Impact	1	0	1	0	19	31	0
Noise	20	0	0	0	0	7	0

Table C.9: Confusion matrix, OOD-2, H4†, $k = 2$.

True \ predicted	Disc.	Cred.	Coll.	Exfil.	Impact	Noise	other
Discovery	18	0	0	0	0	3	0
Credential Access	0	6	0	0	0	0	0
Collection	4	0	14	0	6	3	0
Exfiltration	1	0	0	9	0	2	0
Impact	1	0	4	0	16	31	0
Noise	19	0	1	0	0	7	0

CHAPTER D

ADDITIONAL CLAIM CHECKS

Further rows of the claim checklist, complementing Table 4.15.

Table D.1: Additional claims and permitted readings.

Claim	Evidence	Permitted reading
Qwen is hurt by wrong context	Dev: -0.058 to -0.089 in the heavier conditions; 50% mixes contain 0	Exploratory, on Dev only; the Rule changed by at most 0.016; conditions not ranked.
The Rule has better evidence	Holdout Jaccard, Qwen − Rule: -0.067 $[-0.129, -0.015]$ at $k = 2$; -0.010 $[-0.074, +0.044]$ at $k = 4$	True at $k = 2$ only, against auto-generated ground truth; nominal.
Qwen has better evidence on unseen techniques	OOD-2: $+0.055$ and $+0.092$ (exclude 0); OOD-1 $k = 4$: $+0.095$	Higher Jaccard on OOD-2; not a ranking of explanation quality; OOD-1 not independent.
OOD-1 is an independent validation	Written by Claude; SQL visible after the freeze	Not supported; exploratory results.
Hybrid H3 helps	-0.004 to -0.006 against the Rule on the Holdout, interval contains 0	No evidence of improvement; this does not prove that H3 is useless.

CHAPTER E

ORIGINAL OUTPUTS OF THE STATISTICAL SCRIPTS

The unedited outputs are kept in the project repository and were copied, not recomputed, for this report: `rq_checks_output.txt` (run 20261028_025528: effect of k, drops, differences of drops, fixed-class sensitivity, share of resamples without a class, Noise counts), `rq_extras_output.txt` (run 20261028_061053: H1 baseline, absolute differences, evidence Jaccard, 10 000 resamples, seed 42), and `rq3_multiplicity.json` (max-$|t|$, critical value 2.5922, six rows). The multiplicity results appear in Table 4.11. The resampled class-absence shares in `rq_checks_output.txt` are: Credential Access absent in 35.1% of OOD-1 resamples and in at most 5.3% for every other class and set.

CHAPTER F

REPRODUCTION COMMANDS AND SEEDS

Seeds: 42 for the split, the language model, the bootstrap, and the multiplicity procedure; 2026 for the Holdout generator. Language model: Qwen3 8B (tag `qwen3:8b`), temperature 0, thinking off, Ollama server 0.40.11 with the Python client `ollama==0.6.3` (also pinned in `requirements.txt`: psycopg 3.2.13, pandas 2.3.3, scikit-learn 1.9.1, pytest 8.4.2). Commands (run by the authors where the data are sealed):

- `python -m src.evaluation.verify_rq1` (independent recomputation of RQ1; no differences)
- `python -m src.evaluation.rq_checks` (effect of k, ΔF_1, D, fixed-class sensitivity, Noise counts)
- `python -m src.evaluation.rq_extras` (H1, absolute differences, evidence differences)
- `python -m src.evaluation.rq_multiplicity` (six-comparison family)
- `python -m src.evaluation.final_tables` (builds the 366-row results table)
- `python -m src.evaluation.report_tables` (confusion matrices, Noise counts, novelty beside accuracy)
- `pytest` (all tests; one older test of the OOD-2 scenario file fails and was not fixed because the file is sealed)

CHAPTER G

PROMPT STRUCTURE

The frozen prompt B+C+E is built by `src/models/llm_local.py` and was sent as one user message. The text below is the exact output of the prompt builder for a window with $k = 2$, in which the window itself is replaced by placeholders; the examples come from the generic seed set and from the Dev set only. At $k = 0$ the six seed examples that contain context and all four balancing examples are left out, so that three examples remain; at $k \geq 2$ all ten are shown. Generation used seed 42, temperature 0, thinking off, and a limit of 128 new tokens.

```
You are a PostgreSQL security analyst. Given a target SQL query and optional context (previous queries
↪  in the same session), classify the target query into one of these attack stages:

- Discovery: reconnaissance: listing tables, columns, roles, settings, version or server info
- Credential Access: ONLY PostgreSQL's own authentication data (pg_shadow, pg_authid, role password
↪  hashes) and login attempts. Credentials stored in ordinary application tables are NOT Credential
↪  Access
- Collection: reading data from ordinary tables that hold sensitive information (SELECT * dumps,
↪  password, secret, token, key or personal-data columns), also through PREPARE/EXECUTE, and reading
↪  server files (pg_read_file)
- Exfiltration: exporting data out (COPY TO, lo_export)
- Impact: destructive actions, privilege changes (DROP, ALTER ROLE, CREATE ROLE)
- Noise: normal/benign database activity

A DO block whose EXECUTE content is hidden ('?') cannot be read: judge it by the context, as the same
↪  kind of work as the latest relevant query before it, and call it Impact only if destructive
↪  statements are visible.

Respond ONLY with a JSON object: {"label": "<stage>", "evidence_ids": [<ids>]}
where evidence_ids lists the query IDs (q0, q1, ...) that support your classification. Always include
↪  the target query's ID in evidence_ids. Output ONLY valid JSON, no other text.

Classify ONLY the TARGET query. The context is there to disambiguate, not to carry its stage over: a
↪  harmless query stays Noise even when earlier queries look malicious.

### Examples

Queries:
```

```
TARGET (classify only this query):
q0: SELECT version()
Answer: {"label": "Discovery", "evidence_ids": [0]}

Queries:
CONTEXT (earlier queries, reference only, do NOT classify these):
q0: SELECT table_name FROM information_schema.tables WHERE table_schema = '?'
TARGET (classify only this query):
q1: SELECT * FROM t1
Answer: {"label": "Collection", "evidence_ids": [0, 1]}

Queries:
CONTEXT (earlier queries, reference only, do NOT classify these):
q0: SELECT version()
q1: SELECT rolname, rolsuper FROM pg_shadow
TARGET (classify only this query):
q2: ALTER ROLE postgres PASSWORD '?'
Answer: {"label": "Impact", "evidence_ids": [1, 2]}

Queries:
CONTEXT (earlier queries, reference only, do NOT classify these):
q0: SELECT table_name FROM information_schema.tables WHERE table_schema = '?'
q1: SELECT * FROM t1
TARGET (classify only this query):
q2: COPY (SELECT * FROM t1) TO '?'
Answer: {"label": "Exfiltration", "evidence_ids": [0, 1, 2]}

Queries:
TARGET (classify only this query):
q0: SELECT now()
Answer: {"label": "Noise", "evidence_ids": [0]}

Queries:
TARGET (classify only this query):
q0: SELECT rolname, rolsuper, rolcanlogin FROM pg_shadow WHERE rolcanlogin = true
Answer: {"label": "Credential Access", "evidence_ids": [0]}

Queries:
CONTEXT (earlier queries, reference only, do NOT classify these):
q0: SELECT id, username, email FROM t1 WHERE username = '?'
q1: SELECT count(*) FROM t2 WHERE username = '?' AND read_at IS NULL
TARGET (classify only this query):
q2: SELECT t.title, t.status, t.priority FROM t3 t WHERE t.project_id = ? AND t.status = '?'
Answer: {"label": "Noise", "evidence_ids": [2]}

Queries:
CONTEXT (earlier queries, reference only, do NOT classify these):
q0: SELECT table_name FROM information_schema.tables WHERE table_schema = '?'
TARGET (classify only this query):
q1: SELECT id, username FROM t1 ORDER BY created_at DESC LIMIT ?
```

```
Answer: {"label": "Noise", "evidence_ids": [1]}

Queries:
CONTEXT (earlier queries, reference only, do NOT classify these):
q0: SELECT rolname, rolsuper, rolcanlogin FROM pg_shadow WHERE rolcanlogin = true
TARGET (classify only this query):
q1: SELECT now()
Answer: {"label": "Noise", "evidence_ids": [1]}

Queries:
CONTEXT (earlier queries, reference only, do NOT classify these):
q0: SELECT * FROM t1
TARGET (classify only this query):
q1: DROP TABLE t2
Answer: {"label": "Impact", "evidence_ids": [1]}

### Now classify this:

Queries:
CONTEXT (earlier queries, reference only, do NOT classify these):
q0: <context query 0>
q1: <context query 1>
TARGET (classify only this query):
q2: <target SQL>

Answer:
```

CHAPTER H

AUDIT FINDINGS

The results were audited before this report was written. The 39 findings are summarized; the original table is in `rq_results_review.md`.

#	Type	Finding and handling
1	Reporting	Sign of ΔF_1 was inconsistent; fixed as known minus unseen, positive = drop.
2	Statistical	ΔF_1 compared different class sets; recomputed on shared classes.
3	Statistical	$k = 0$ differs from $k \geq 2$ in number of examples (3 vs 10); stated in the text.
4	Reporting	"Context helps at $k = 2$ lacked paired intervals; all contain 0.
5	Reporting	Corrupted-context loss was on Dev, not Holdout; reported per condition with intervals.
6	Confirmed error	Claim that the advantage is on techniques used in Dev was wrong: reused families reuse OOD-1 techniques.
7	Confirmed error	H4 was not pre-registered from the start; idea post hoc, definition fixed before OOD-2.
8	Confirmed error	"One reviewer hid that the reviewer is an AI; stated explicitly.
9	Reporting	C2 had been described as lightest; it has the most records.
10	Reporting	"Five of six attributes contradicted the table; per-field rates only.
11	Reporting	"LLM more robust, significant corrected to the six-comparison result.
12	Reporting	Two Noise counts for Qwen on OOD-1 were for different k (6, 5, 4).
13	Reporting	Missing intervals for Qwen and H4 on OOD-2 added.
14	Statistical	Paired evidence intervals added.
15	Statistical	H1 compared two different sets of windows; redefined as four numbers.
16	Statistical	Holdout-versus-OOD difference is not novelty alone.
17	Statistical	Scenario bootstrap does not cover between-family uncertainty.
18	Reporting	Causal wording in the ablation toned down.
19	Reporting	Dashboard labels reworded to "interval excludes 0 / insufficient evidence.
20	Reporting	Dev-to-Holdout fall is not evidence of no leakage.
21	Statistical	Class set varied between resamples; fixed-class estimator added.
22	Confirmed error	Script printed incomplete blocks; fixed and rerun.
23	Reporting	Hand-typed RQ1 numbers in the dashboard replaced by computed values.
24	Reporting	Dashboard showed default-estimator ΔF_1; main table now fixed-class.
25	Reporting	Hand-typed numbers in dashboard text replaced by computed ones.
26	Confirmed error	Forest chart axis ticks cut intervals; fixed.
27	Confirmed error	Page heading hidden by navigation bar; fixed.
28	Reporting	Charts unreadable on narrow screens; fixed.
29	Reporting	Dev results had been more prominent than Holdout; reordered.
30	Reporting	H4 missing for Holdout; added by `rq_extras`.
31	Statistical	Absolute differences used the default estimator; fixed-class version added.
32	Confirmed error	Older OOD-2 scenario test fails; not fixed (sealed file).

#	Type	Finding and handling
33	Confirmed error	Verdict labels for evidence and H1 were wrong in the dashboard; fixed.
34	Reporting	H1 difference read as system accuracy; redefined.
35	Reporting	Dashboard used nominal intervals while claiming adjustment; fixed.
36	Reporting	H4 missing for OOD-1; added afterwards and marked †.
37	Confirmed error	ΔF_1 definition accidentally deleted in the dashboard; restored.
38	Reporting	Overlapping intervals do not decide a comparison; reading guide added.
39	Reporting	136 actions (RQ1) versus 102 queries (RQ2) are different units, not a discrepancy.